

第 12 章 PCIe 总线的应用

本章以分析一个 EP 的硬件设计原理和基于这个 EP 的 Linux 驱动程序为线索，说明 PCIe 设备和基于该设备的 Linux 驱动程序的设计流程。本章使用的 PCIe 设备基于 Xilinx 公司 Vetex-5 XC5VLX50T 内嵌的 PCI Express EP 模块，该模块也被 Xilinx 称为 LogiCORE。

LogiCORE 提供了 PCIe 设备的物理层和数据链路层，并做了一些基本的与事务层相关的工作，这使得许多设计者在并不完全清楚 PCIe 体系结构的情况下，也可以设计实现具有 PCIe 总线接口的设备。本章所述的 PCIe 设备基于 LogiCORE，本章将该 PCIe 设备简称为 Capric 卡。

12.1 Capric 卡的工作原理

Capric 卡的组成结构如图 12-1 所示。

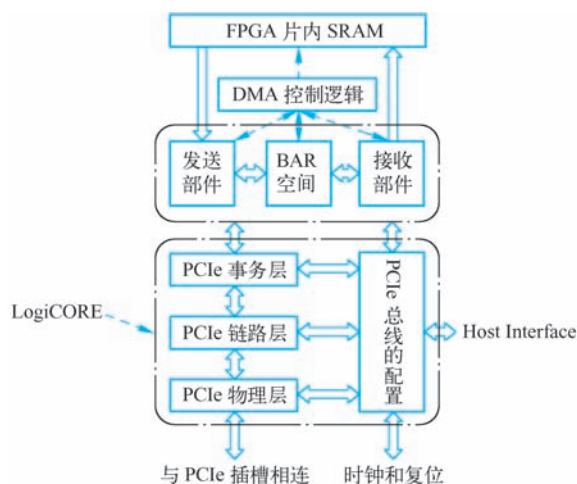


图 12-1 Capric 卡的组成结构

Capric 卡基于 PCIe 总线，主要功能是通过 DMA 读写方式与 HOST 处理器进行数据交换。Capric 卡由 LogiCORE、发送部件、接收部件、BAR 空间、DMA 控制逻辑和 FPGA 片内 SRAM 组成，其工作原理较为简单。

Capric 卡首先使用 DMA 读方式，将主存储器中的数据搬移到 FPGA 的片内 SRAM 中，然后使用 DMA 写方式，将 FPGA 的片内 SRAM 中的数据写入主存储器中。在 Capric 卡中，一次 DMA 操作可以传送的数据区域的最大值为 0x7FFB(0x2047B)。

Capric 卡的各个组成模块的功能描述如下所示。

- LogiCORE。其主要功能是处理 PCIe 设备的物理层、链路层与部分事务层的逻辑，并向外提供必要的接口。PCIe 设备配置空间的初始化，以及与配置和中断请求相关的总线事务也由 LogiCORE 完成。LogiCORE 是 PCIe 总线的接管者，其他部件通过 Log-

iCORE 与 PCIe 链路进行通信。LogiCORE 通过“Host Interface”实现 PCIe 设备的初始化配置。

- Capric 卡的发送部件负责发送 TLP 报文，包括“存储器读请求”和“存储器写请求 TLP”，但是并不包含配置和消息报文的发送。MSI 报文由发送部件通过 LogiCORE 发送。
- 接收部件负责接收“存储器读完成 TLP”。Capric 卡不支持 I/O 读写 TLP。
- DMA 控制逻辑协调发送与接收部件，以完成 DMA 写与 DMA 读操作，该逻辑的实现是 Capric 卡的设计重点。
- BAR 空间中存放了一组操纵 DMA 控制逻辑的寄存器，这组寄存器由 HOST 处理器和 Capric 卡共同读写，从而完成相应的 DMA 操作。Capric 卡仅使用了 BAR0 空间，处理器使用存储器映像寻址方式，而不是 I/O 映像寻址方式访问 BAR0 空间。

12.1.1 BAR 空间

Capric 卡仅使用 BAR0 空间，其大小为 256B，在该空间中包含以下寄存器，这些寄存器使用小端编码方式，如表 12-1 所示。

表 12-1 Capric 卡的 BAR 空间寄存器

缩 写	偏 移	描 述
DCSR1 (Device Control and Status Register 1)	0x00	设备控制和状态寄存器 1
DCSR2 (Device Control and Status Register 2)	0x04	设备控制和状态寄存器 2
WR_DMA_ADR	0x08	DMA 写地址寄存器
WR_DMA_SIZE	0x0C	DMA 写传送大小寄存器
RD_DMA_ADR	0x1C	DMA 读地址寄存器
RD_DMA_SIZE	0x20	DMA 读传送大小寄存器
INT_REG	0x2C	中断状态寄存器
ERR	0x30	错误状态寄存器

(1) DCSR1 寄存器，该寄存器由 7 个有效位组成。

- init_rst_o，第 0 位，该位可读写，复位值为 0。为 1 表示 Capric 卡处于复位状态，为 0 表示 Capric 卡已经完成复位。软件通过操纵该位对 Capric 卡进行复位。其过程为首先向此位写 1，然后延时至少 5 μ s 后（Capric 卡内逻辑和 FPGA 的片内 SRAM 需要至少 5 μ s 的复位时间），再向此位写 0。
- int_rd_enb，第 8 位，该位可读写，复位值为 0。为 1 表示当 DMA 读完成后，Capric 卡可以向处理器提交中断请求；为 0 表示 DMA 读完成后，Capric 卡不能向处理器提交中断请求。该位为 DMA 读完成中断使能位。
- int_wr_enb，第 9 位，该位可读写，复位值为 0。为 1 表示当 DMA 写完成后，Capric 卡可以向处理器提交中断请求；为 0 表示 DMA 写完成后，Capric 卡不能向处理器提交中断请求。该位为 DMA 写完成中断使能位。
- int_rd_msk，第 16 位，该位可读写，复位值为 0。为 1 表示当 DMA 读完成后，Capric

卡不能向处理器提交中断请求，而是置 `int_rd_pending` 位为 1；为 0 表示 DMA 读完成后，Capric 卡可以向处理器提交中断请求。该位为 DMA 读完成中断屏蔽位。

- `int_rd_pending`，第 17 位，该位可读写，复位值为 0。为 1 表示 Capric 卡含有未发出的 DMA 读完成中断请求，当 `int_rd_msk` 位由 1 变为 0 时，Capric 卡发送该中断请求；为 0 表示 Capric 卡不含有未发出的 DMA 读完成中断请求。
- `int_wr_msk`，第 24 位，该位可读写，复位值为 0。为 1 表示当 DMA 写完成后，Capric 卡不能向处理器提交中断请求，而是置 `int_wr_pending` 位为 1；为 0 表示 DMA 写完成后，Capric 卡可以向处理器提交中断请求。该位为 DMA 写完成中断屏蔽位。
- `int_wr_pending`，第 25 位，该位可读写，复位值为 0。为 1 表示 Capric 卡含有未发出的 DMA 写完成中断请求，当 `int_wr_msk` 位由 1 变为 0 时，Capric 卡发送该中断请求；为 0 表示 Capric 卡不含有未发出的 DMA 写完成中断请求。

设置 Mask 和 Pending 位的主要目的是为了防止中断丢失和产生 Spurious 中断请求，这两位与 MSI Capability 结构中的 Mask 和 Pending 位的功能相似。LogiCORE 并不支持 MSI Capability 结构中的 Mask 和 Pending 位，因此 Capric 卡引入了这两个位。

(2) `DCSR2`，该寄存器由 4 位组成。分别为 `mwr_start`，DMA 写启动位；`mwr_done`，DMA 写结束位；`mrd_start`，DMA 读启动位；`mrd_done`，DMA 读结束位。

- `mwr_start`，第 0 位，该位可读写，复位值为 0。系统软件向该位写 1 时启动 DMA 写操作，软件对此位写 0 无意义。
- `wr_done_clr`，第 1 位，可读，写 1 清除。当一次 DMA 写完成后，`wr_done_clr` 位将置 1，系统软件将 `wr_done_clr` 位写 1 清零后，`mwr_start` 位也将清零，之后系统软件可以重新启动下一次 DMA 写操作。
- `mrd_start`，第 16 位，该位可读写，复位值为 0。系统软件向该位写 1 时启动 DMA 读操作，软件对此位写 0 无意义。
- `rd_done_clr`，第 17 位，可读，写 1 清除。当一次 DMA 读完成后，`rd_done_clr` 位将置 1，系统软件将 `rd_done_clr` 位写 1 清零后，`mrd_start` 位也将清零，之后系统软件可以重新启动下一次 DMA 读操作。

(3) `WR_DMA_ADR`，DMA 写地址寄存器，该寄存器存放 DMA 写操作的目的地址，该寄存器的复位值无意义。再一次强调该寄存器存放的地址为 PCI 总线域的地址，而不是存储器域的地址，尽管在许多处理器系统中，该地址的 PCI 总线域地址与存储器域地址相等。该寄存器由 32 位组成，存放 32 位的 PCI 总线域地址。

(4) `WR_DMA_SIZE`，DMA 写传送大小寄存器。该寄存器存放一次 DMA 写操作的传送大小，以字节为单位。该寄存器的复位值为 0，由 32 位组成，但是只有低 11 位有效，因此 Capric 卡一次 DMA 传送的最大值为 0x7FF。该寄存器为 N 时表示一次 DMA 写操作的传送大小为 N 个字节。该寄存器为 0 时，即便 `DCSR2` 寄存器的 `mwr_start` 位为 1 时，也不能启动 DMA 写传送。当 `mwr_start` 位为 1，该寄存器由 0 变为其他数据时，也将启动 DMA 写传送。

(5) `RD_DMA_ADR`，DMA 读地址寄存器，该寄存器存放 DMA 读操作的目的地址，该寄存器的复位值无意义。该寄存器存放的地址为 PCI 总线域地址，由 32 位组成。Capric 仅支持 32 位的 PCI 总线地址。

(6) RD_DMA_SIZE, DMA 读传送大小寄存器, 存放一次 DMA 读操作的传送大小, 以字节为单位。该寄存器的复位值为 0, 由 32 位组成, 但是只有最低 11 位有效, 因此 Capric 卡一次 DMA 读传送的最大值为 0x7FF。该寄存器为 N 时表示一次 DMA 读操作的传送大小为 N 个字节。该寄存器为 0 时, 即便 DCSR2 寄存器的 mrd_start 位为 1 时, 也不能启动 DMA 读传送。当 mrd_start 位为 1 时, 该寄存器由 0 变为其他数据时, 将启动 DMA 读传送。

(7) INT_REG, 中断控制状态寄存器, 该寄存器存放 Capric 卡的中断状态, 该寄存器共由 5 个有效位组成。

- int_src_rd, 第 0 位, 该位只读, 复位值为 0。当 Capric 卡的 DMA 读操作完成后, 而且 DCSR1 寄存器的 int_rd_enb 位为 1 时, 该位为 1 表示 Capric 卡已经向处理器提交了 DMA 读完成中断请求。值得注意的是当 int_rd_msk 位为 1 时, Capric 卡不能发送 DMA 读完成中断, 此时 int_src_rd 位需要等待 Capric 卡置 int_rd_msk 位为 0, 发送完毕 DMA 读完成中断后, 才能置 1。
- int_src_wr, 第 1 位, 该位只读, 复位值为 0。当 Capric 卡的 DMA 写操作完成后, 而且 DCSR1 寄存器的 int_wr_enb 位为 1 时, 该位为 1 表示 Capric 卡已经向处理器提交了 DMA 写完成中断请求。值得注意的是当 int_wr_msk 位为 1 时, Capric 卡不能发送 DMA 写完成中断, 此时 int_src_wr 位需要等待 Capric 卡置 int_wr_msk 位为 0, 发送完毕 DMA 写完成中断后, 才能置 1。
- rd_done_clr, 第 8 位, 该位可读写, 复位值为 0, 该位为 1 表示 DMA 读操作完成。DMA 读结束后该位由硬件置 1, 软件将该位清零后, 硬件才能进行下一次 DMA 读操作。该位置 1 后, 如果 DCSR1 寄存器的 int_rd_enb 位为 1 时, Capric 卡将向处理器提交中断请求[⊖]。

系统软件在中断服务例程中, 向该位写 1 将清除该位; 向该位写 0 无意义。如果 Capric 卡不使用中断方式接收数据, 系统软件可以查询该位确定当前 DMA 读操作是否已经完成。此位与 DCSR2 寄存器的 rd_done_clr 位相同, 软件清除 INT_REG 寄存器的该位后, DCSR2 寄存器的 rd_done_clr 位也被清零。

- wr_done_clr, 第 9 位, 该位可读写, 复位值为 0, 该位为 1 表示 DMA 写操作完成。DMA 写完成后该位由硬件置 1, 软件将该位清零后, 硬件才能进行下一次 DMA 写操作。该位置 1 后, 如果 DCSR1 寄存器的 int_wr_enb 位为 1 时, Capric 卡将向处理器提交中断请求。

系统软件在中断服务例程中, 向该位写 1 将清除该位; 向该位写 0 无意义。如果 Capric 卡不使用中断方式接收数据, 系统软件可以查询该位确定当前 DMA 写操作是否已经完成。此位与 DCSR2 寄存器的 wr_done_clr 位相同, 软件清除 INT_REG 寄存器的该位后, DCSR2 寄存器的 wr_done_clr 位也被清零。

- int_asserted, 第 31 位, 该位只读, 复位值为 0。当 Capric 卡向处理器提交中断请求时, 该位由硬件逻辑置 1。对此位进行写操作无意义。在逻辑实现中, $\text{int_asserted} = \text{int_src_rd} \& \text{int_src_wr}$ 。

⊖ Capric 卡可以使用 Legacy INTx 或者 MSI 方式进行中断请求。

12.1.2 Capric 卡的初始化

Capric 卡在初始化时需要进行配置寄存器空间和 Capric 卡硬件逻辑的初始化。其中配置寄存器空间的初始化由软硬件联合完成。

Capric 卡的设计基于 Xilinx 公司的 LogiCORE。因此 Capric 卡需要使用 Xilinx 公司提供的“CORE Generator GUI”对 LogiCORE 进行基本的初始化，并设置一些必要的参数，包括 Vendor ID、Device ID、Revision ID 和 Subsystem ID 等参数。有关该工具的使用见 [LogiCORE(tm) Endpoint PIPE v1.7]，本节对此不做进一步描述。Capric 卡的配置寄存器空间的初始值如下所示。

- Vendor ID 为 0x10EE，Xilinx 使用的 Vendor ID。
- Device ID 为 0x0007，LogiCORE 使用的 Device ID。
- Revision ID 为 0x00。
- Subsystem ID 为 0x10EE。
- Device ID 为 0x0007。
- Base Class 为 0x05，表示 Capric 卡为“类存储器控制器”。
- Sub Class 和 Interface 为 0x00，进一步描述 Capric 卡为 RAM 控制器。
- Card CIS Pointer 为 0x00，表示不支持 Card Bus 接口。
- BAR0 为 0xFFFFFFFF00。Capric 卡仅支持 BAR0 空间，该空间采用 32 位存储器映像寻址，其大小为 256B，而且不支持预读。在初始化时，BAR0 寄存器存放该空间所需要的存储器空间大小，该寄存器由系统软件读取后，再写入一个新的数值。这个数值为 BAR0 空间使用的基地址。
- Max_Payload_Size Supported 参数为 0b010，即 Max_Payload_Size Supported 参数的最大值为 512B。多数 RC 支持的 Max_Payload_Size Supported 参数仅为 128B 或者 256B。因此 LogiCORE 支持 512B 已经足够了。在 Capric 卡的初始化阶段，需要与对端设备进行协商，确认 Max_Payload_Size 参数的值，如果 Capric 卡与 Intel 的 Chipset 直接相连，该参数为 128B 或者 256B。Capric 卡需要根据协商后的 Max_Payload_Size 参数，而不是 Max_Payload_Size Supported 参数，确定存储器写 TLP 有效负载的大小。当 DMA 写的数据区域超过 Max_Payload_Size 参数时，需要进行拆包处理，详见第 12.2.1 节。
- Capric 卡不支持 Phantom 功能。即不能使用 Function 号，进一步扩展 Tag 字段。Phantom 功能的详细说明见第 4.3.2 节。
- Multiple Message Capable 参数为 0b000，即支持一个中断向量。
- Max_Read_Request_Size 参数为 0b010，即存储器读请求 TLP 一次最多能够从目标设备中读取 512B 大小的数据。如果 DMA 读的数据区域超过 512B 时，需要进行拆包处理，详见第 12.2.2 节。

系统软件在 Capric 卡初始化时，将分析 Capric 卡的配置空间，并填写 Capric 卡的配置寄存器空间。值得注意的是，系统软件对 Capric 卡进行配置时，Capric 卡将保留该设备在 PCI 总线树中的 Bus Number、Device Number 和 Function Number，LogiCORE 使用寄存器 `cfg_bus_number[7:0]`、`cfg_device_number[4:0]` 和 `cfg_function_number[2:0]` 存放这组数值，当 LogiCORE 发起存储器读请求 TLP 时，需要使用这组数值。

在设备驱动程序中，Capric 卡需要执行以下步骤完成硬件初始化。

- 1) 向 DCSR1 寄存器的 `init_rst_o` 位写 1。
- 2) 延时 5 μ s。
- 3) 向 DCSR1 寄存器的 `init_rst_o` 位写 0。
- 4) 向 DCSR1 寄存器的 `int_rd_enb` 和 `int_wr_enb` 位写 1，使能 DMA 读写中断请求。

12.1.3 DMA 写

Capric 卡使用 DMA 写过程将 Capric 卡 SRAM 中的数据发送到 HOST 处理器。在设备驱动程序中，DMA 写过程如下所示。

- 1) 填写 `WR_DMA_ADR` 寄存器，注意填写的是 PCI 总线域的地址。
- 2) 填写 `WR_DMA_SIZE` 寄存器，以字节为单位。
- 3) 填写 DCSR2 寄存器的 `mwr_start` 位，启动 DMA 写。
- 4) 等待 DMA 写完成中断后，结束 DMA 写。如果系统软件屏蔽了 DMA 写完成中断，可以通过查询 `INT_REG` 寄存器的 `wr_done_clr` 位判断 DMA 写是否已经完成。在 Capric 卡中，上一次 DMA 写操作没有完成之前，不能启动下一次 DMA 写操作。
- 5) 最后将 `wr_done_clr` 位清零。

从硬件设计的角度来看，DMA 写过程较为复杂。Capric 卡需要通过 DMA 控制逻辑，组织一个或者多个存储器写 TLP，将 SRAM 中的数据进行封装然后传递给发送部件，再由发送部件将数据传送到 LogiCORE，最后由 LogiCORE 将存储器写 TLP 传递给 RC。

如果一次 DMA 写所传递的数据超过了 512B[⊖]，那么 DMA 控制逻辑需要传递多个存储器写 TLP 给发送部件，才能完成一次完整的 DMA 写操作。而且在 DMA 操作中需要进行数据对界。其详细实现过程见第 12.2.1 节。

12.1.4 DMA 读

Capric 卡使用 DMA 读过程将主存储器中的数据读到 Capric 卡片内 SRAM 中。在设备驱动程序中，DMA 读过程如下所示。

- 1) 填写 `RD_DMA_ADR` 寄存器，注意此处填写的是 PCI 总线域的地址。
- 2) 填写 `RD_DMA_SIZE` 寄存器，以字节为单位。
- 3) 填写 DCSR2 寄存器的 `mrd_start` 位，启动 DMA 读。
- 4) 等待 DMA 读完成中断产生后，结束 DMA 读。如果系统软件屏蔽了 DMA 读完成中断，则系统软件可以通过查询 `INT_REG` 寄存器的 `rd_done_clr` 位判断是否 DMA 读已经完成。在 Capric 卡中，上一次 DMA 读操作没有完成之前，不能启动下一次 DMA 读操作。
- 5) 最后将 `rd_done_clr` 位清零。

从硬件设计的角度来看，DMA 读过程比 DMA 写过程复杂。PCIe 总线使用 Split 方式实现存储器读。Capric 卡的 1 次 DMA 读操作使用两种 TLP 报文，并通过发送部件和接收部件协调完成。

[⊖] LogiCORE 规定 `Max_Payload_Size` 为 512B。

1) 首先 DMA 控制逻辑组织一个或者多个存储器读请求 TLP，然后由发送部件将存储器读请求 TLP 传递给 RC。

2) RC 正确接收到这个请求报文后，使用一个或者多个存储器读完成 TLP 将数据传递给 Capric 卡。

3) Capric 卡接收逻辑从 RC 中获得这些存储器读完成 TLP 时，需要首先处理乱序，之后完成一个 DMA 读操作，并向 RC 提交中断请求。

如果一次 DMA 读请求的数据大于 512B^①时，DMA 控制逻辑需要发送多个存储器读请求 TLP 给 RC，而且在 DMA 读操作中需要进行数据对界。尤其值得注意的是这几个 TLP 的 Tag 字段不能相同，为此硬件逻辑必须正确维护存储器读请求使用的 Tag 字段。DMA 读操作的详细实现过程见第 12.2.2 节。

12.1.5 中断请求

Capric 卡使用 MSI 机制提交中断请求，并只使用了一个中断向量处理 DMA 读/写完成和错误处理。本章为简便起见，忽略了错误处理的过程，但是在一个实际的设计中，错误处理及恢复过程非常重要。

当 DCSR1 寄存器的 int_rd_enb 和 int_wr_enb 位为 1，而且 int_wr_msk 和 int_rd_msk 不为 1 时，DMA 读写完成后，Capric 卡将向处理器提交中断请求。当 DMA 读写完成后，硬件逻辑将 INT_REG 寄存器的 int_asserted 位置为 1，表示有中断请求。此时系统软件需要进一步查询 INT_REG 寄存器的 int_src_rd 和 int_src_wr 位^②，判断该中断请求为 DMA 读完成还是 DMA 写完成，其步骤如下。

1) 如果 int_asserted 位为 1，表示 Capric 卡提交了一个中断请求；否则转 6)。

2) int_src_rd 位为 1，表示 Capric 卡提交了一个 DMA 读完成中断请求，否则转 4)。此时 rd_done_clr 位也应该为 1。

3) 进行 DMA 读完成处理。向 rd_done_clr 位写 1，清除 DMA 读完成请求位，转 6)。

4) 如果 int_src_wr 位为 1，表示 Capric 卡提交了一个 DMA 写完成中断请求。此时 wr_done_clr 位也应该为 1。

5) 进行 DMA 写完成处理。向 wr_done_clr 位写 1，清除 DMA 读完成请求位。

6) 结束。

以上过程仅为一个简单的中断服务例程的执行流程，一个具体设备驱动程序在 DMA 读写完成后，将检查一些返回状态，以确定 DMA 读写是否正确结束。

12.2 Capric 卡的数据传递

本节主要介绍系统软件在启动 DMA 读写操作时，硬件逻辑的工作过程，包括软件启动 DMA 写时，Capric 卡如何向 RC 发送存储器写 TLP；软件启动 DMA 读时，Capric 卡如何向

① LogiCORE 规定 Max_Read_Request_Size 为 512B。

② Capric 卡也可以使用多个 MSI 报文 (Multiple Message)，其中 DMA 写完成和读完成分别对应一个 MSI 报文，而无须查询，目前 Linux 系统并不支持 PCIe 设备的 Multiple Message 功能。

RC 发送存储器读请求 TLP，以及 Capric 卡如何接收来自 RC 的存储器读完成 TLP。

如果考虑 DMA 读写操作的数据对界，DMA 读写操作的实现较为复杂。为此本节定义了两种操作处理对界问题。

(1) 向前 X 字节对界

向前对界操作使用 $\text{Head}_X(Y)$ 函数，其中 Y 参数对应某个物理地址；而 X 参数对应对界单位，其值必须为 2 的幂。该函数的计算方法如式 12-1 所示。

$$\text{Head}_X(Y) = Y - (Y \bmod X) \quad (12-1)$$

由以上公式，可以得出 $\text{Head}_4(0x1000) = 0x1000$ ，而 $\text{Head}_4(0x1007) = 0x1004$ 。该操作非常适合硬件实现，在硬件实现中 $\text{Head}_X(Y) = Y_n Y_{n-1} \cdots Y_m 0_{m-1} 0_{m-2} \cdots 0_1 0_0$ 。如果 Y 的长度为 32 bit，则 n 等于 31，而 m 等于 $\log_2(X)$ 。因此在硬件逻辑中，只要将 Y 的第 $0 \sim m-1$ 位清零即可。

(2) 向后 X 字节对界

向后对界操作使用 $\text{Tail}_X(Y)$ 函数，其中 Y 参数对应某个物理地址；而 X 参数对应对界单位，其值必须为 2 的幂。该函数的计算方法如式 12-2 所示。

$$\text{Tail}_X(Y) = \text{Head}_X(Y) + X - 1 \quad (12-2)$$

由以上公式，可以得出 $\text{Tail}_4(0x1000) = 0x1003$ ，而 $\text{Tail}_4(0x1007) = 0x1007$ 。该操作也非常适合硬件实现，在硬件实现中 $\text{Tail}_X(Y) = Y_n Y_{n-1} \cdots Y_m 1_{m-1} 1_{m-2} \cdots 1_1 1_0$ 。如果 Y 的长度为 32 bit，则 n 等于 31，而 m 等于 $\log_2(X)$ 。因此在硬件逻辑中，只要将 Y 的第 $0 \sim m-1$ 位置 1 即可。

12.2.1 DMA 写使用的 TLP

当软件启动 DMA 写过程后，DMA 控制逻辑将组织存储器写 TLP 发送给 RC。PCIe 总线使用 Posted 总线事务发送存储器写 TLP。Capric 卡使用 4DW 长度的 TLP 头，即使用 64 位地址编码格式。存储器写 TLP 由一个通用 TLP 头加上若干数据字段组成，存储器写 TLP 格式如图 12-2 所示。

	+0				+1				+2				+3			
	7	6	5	4 ~ 0	7	6 ~ 4	3 ~ 0		7	6	5 ~ 4	3 ~ 2	1 ~ 0		7 ~ 0	
Byte 0	R	Fmt		Type	R	TC		R	$\frac{1}{2}$	$\frac{1}{2}$	Attr	AT	Length			
Byte 4	与 Type 字段相关												Last DW BE	First DW BE		
Byte 8	Address[63:32]															
Byte 12	Address[31:2]														R	
Byte 16	Data 0															
	...															
Byte ...	Data(Length-1)															

图 12-2 存储器写请求 TLP

1. DMA 写操作使用的实际长度

DMA 写逻辑首先从 WR_DMA_ADR 寄存器中获得起始地址 $A(A_{31}A_{30}\cdots A_1A_0)$ ，然后从 WR_DMA_SIZE 寄存器中获得传送长度 $L(L_{15}L_{14}\cdots L_1L_0)$ ，并由 A 和 L ，通过计算获得本次 DMA 写的结束地址 $B(B_{31}B_{30}\cdots B_1B_0)$ ，其值如式 12-3 所示。

$$B = A + L - 1 \quad (12-3)$$

系统软件需要保证 B 的计算结果不会出现进位，而 DMA 写逻辑由公式 12-3 获得本次数据传送的地址范围 $A \sim B$ 。在存储器读写 TLP 中，Length 字段以 DW 为单位，因此向 $A \sim B$ 这段数据区域进行 DMA 写时，Capric 卡实际上需要向 $\text{Head}_4(A) \sim \text{Tail}_4(B)$ 这段数据区域进行 DMA 写操作，同时使用 TLP 的 First DW BE 和 Last DW BE 字段进行对界处理。

如果一次 DMA 写操作向 $0\text{xFFF0}-0003 \sim 0\text{xFFF0}-0200$ 这段区域传送数据时，虽然这段数据区域的长度为 0x1FE 字节^①，但是由于 TLP 的 Length 字段以 DW 为基本单位，因此 Capric 卡需要向 $0\text{xFFF0}-0000 \sim 0\text{xFFF0}-0203$ 这段区域进行写操作，然后通过 First DW BE 和 Last DW BE 字段屏蔽 $0\text{xFFF0}-0000 \sim 0\text{xFFF0}-0002$ 和 $0\text{xFFF0}-0201 \sim 0\text{xFFF0}-0203$ 这两段数据区域。

因此在 L 中存放的长度 ($A \sim B$ 数据区域的长度) 并不是该 TLP 使用的实际长度。在该 TLP 中使用的实际长度为 $\text{Head}_4(A) \sim \text{Tail}_4(B)$ 这段数据区域的长度。本节使用 $M(M_{15}M_{14}\cdots M_1M_0)$ 保存 TLP 中使用的实际长度，其值如式 12-4 所示。

$$M = (\text{Tail}_4(B) - \text{Head}_4(A) + 1) \gg 2 \quad (12-4)$$

使用以上公式可以较为方便地得出 DMA 写操作使用的实际长度，但是公式 12-3 和公式 12-4 中使用了多个 32 位的加法器，非常耗费 FPGA 的内部资源。为此本次设计使用了另外一种算法计算 M 的值，如式 12-5 所示。

$$\begin{aligned} M' &= [\text{Head}_4(00A_1A_0 + 00L_1L_0 + 3) - \text{Head}_4(00A_1A_0)] \\ M &= L \gg 2 + M' \gg 2 \end{aligned} \quad (12-5)$$

我们可以使用公式 12-5 计算 $0\text{xFFF0}-0003 \sim 0\text{xFFF0}-0200$ 这段区域使用的实际长度 M 。在这段区域中， A 为 $0\text{xFFF0}-0003$ ，而 $L = 0\text{x1FE}$ ，因此 M' 等于 8，而 $M = 0\text{x7F} + 0\text{x2} = 0\text{x81}$ 。该结果与式 12-4 计算结果相等。但是在式 12-5 中， M' 的计算仅使用 4 位加法器，其实现代价比式 12-4 所耗费资源少得多，更为重要的是计算速度也比式 12-4 快得多。

在 LogiCORE 中，Max_Payload_Size Supported 参数的最大值为 512B。但是链路两端经过协商后，实际确认的 Max_Payload_Size 参数可能小于 512B，在多数 x86 处理器系统中，该参数为 128B，因此下文假设 Max_Payload_Size 参数为 128B。

当 M 大于 0x20 （即数据区域的实际长度超过 128B）时，Capric 卡进行 DMA 写时需要发送多个存储器写请求 TLP，而 M 小于或等于 0x20 时仅需要发送 1 个存储器写请求 TLP。下文分别讨论这两种情况。

2. M 小于或等于 0x20

Capric 卡向数据区域 $[A_{31}A_{30}\cdots A_1A_0 \sim B_{31}B_{30}\cdots B_1B_0]$ 进行 DMA 写操作时，如果通过公式 12-5 的计算发现 M 小于或等于 0x20 ，DMA 控制逻辑将组织 1 个或者 2 个存储器写

① 数据区域的大小为数据尾地址-数据首地址+1，即 $0\text{xFFF0}-0200 - 0\text{xFFF0}-0003 + 1 = 0\text{x1FE}$ 。

TLP 传递给 LogiCORE。

如果这个 TLP 所传递的数据区域跨越了 4 KB 边界，将组织 2 个存储器写 TLP，因为 PCIe 总线规定被传送的数据区域不能跨越 4 KB 边界；如果没有跨越 4 KB 边界，Capric 卡组织 1 个存储器写 TLP。我们首先讨论这段数据区域没有跨越 4 KB 边界的情况，此时这个存储器写 TLP 的各个字段如下所示。

- Fmt 字段为 0b10 或者 0b11，表示使用 3DW 或者 4DW 的 TLP 头，而且带有数据。在 Capric 中，Fmt 字段为 0b011。
- Type 字段为 0b00000，表示当前 TLP 为存储器写 TLP。
- TC 字段为 0b000，表示传送类型为 TC0。
- TD 位为 0b0，表示当前 TLP 不含有 ECRC 信息。
- EP 位为 0b0，表示当前 TLP 是正常的，没有出现完整性问题。
- Attr 字段为 0b00，表示当前 TLP 不使用 Relaxed Ordering，由硬件完成 Cache 一致性操作。有关 Cache 一致性的处理见第 3.3 节和第 12.3.6 节，而 Relaxed Ordering 的描述见第 11.4 节。
- AT 字段为 0b00，表示不进行地址转换。
- Length 字段由公式 12-5 计算而来，其值与 M 相等，单位为 DW，最大值为 0x20，即 128B。假定在 Capric 卡中，Max_Payload_Size 参数为 128B。在 x86 处理器系统中，多数 RC 的 Max_Payload_Size 参数为 128B。
- Address 字段为 $A_{31}A_{30}\cdots A_2$ 。Address 字段为 DW 对界的，共由 30 位组成。
- 当 M 不等于 1 时，Last DW BE 字段与 TLP 报文的结束地址有关，更准确地讲，与 B_1 和 B_0 位有关，如下所示。
 $B_1B_0=0b11$ ，则 Last DW BE 字段为 0x1111。
 $B_1B_0=0b10$ ，则 Last DW BE 字段为 0x0111。
 $B_1B_0=0b01$ ，则 Last DW BE 字段为 0x0011。
 $B_1B_0=0b00$ ，则 Last DW BE 字段为 0x0001。
- 当 M 不等于 1 时，First DW BE 字段与 TLP 报文的起始地址有关，更准确地讲，与 A_1 和 A_0 位有关，如下所示。
 $A_1A_0=0b00$ ，则 First DW BE 字段为 0x1111。
 $A_1A_0=0b01$ ，则 First DW BE 字段为 0x1110。
 $A_1A_0=0b10$ ，则 First DW BE 字段为 0x1100。
 $A_1A_0=0b11$ ，则 First DW BE 字段为 0x1000。

当 M 等于 1 时，Last DW BE 字段必须为 0b0000；First DW BE 字段的计算与 A_1A_0 和 B_1B_0 相关，其中 $(\text{First DW BE})A_1A_0 \sim (\text{First DW BE})B_1B_0$ 字段为 1，其他位为 0。这两个字段的关系如表 12-2 所示。

表 12-2 First DW BE 与 A_1A_0 ， B_1B_0 之间的关系

A_1A_0	B_1B_0	First DW BE	Last DW BE
0b00	0b00	0b0001	0b0000
0b00	0b01	0b0011	0b0000
0b00	0b10	0b0111	0b0000

(续)

$A_1 A_0$	$B_1 B_0$	First DW BE	Last DW BE
0b00	0b11	0b1111	0b0000
0b01	0b01	0b0010	0b0000
0b01	0b10	0b0110	0b0000
0b01	0b11	0b1110	0b0000
0b10	0b10	0b0100	0b0000
0b10	0b11	0b1100	0b0000
0b11	0b11	0b1000	0b0000

值得注意的是，当 M 小于或等于 $0x20$ 时，TLP 所传递的报文，其数据区域依然可能会跨越 4 KB 边界。如 Capric 卡向 $0xFFFF-0FFF \sim 0xFFFF-1000$ 数据区域进行 DMA 写时，虽然 M 等于 $0x2$ （实际长度只有两个字节），但是 Capric 卡需要使用两个存储器写请求 TLP，分别向 $0xFFFF-0FFF \sim 0xFFFF-00-0FFF$ 和 $0xFFFF-1000 \sim 0xFFFF-1000$ 这两段数据进行写操作。

由此可以发现，当 Capric 卡对 $A \sim B$ 这段数据区域^①进行 DMA 写时，首先需要判断这段区域是否跨越 4 KB 边界^②。如果跨越则需要向 $A \sim \text{Tail}_{4096}(A)$ 和 $\text{Head}_{4096}(B) \sim B$ 这两段数据区域进行写操作，这两段数据区域一定都小于 $0x20$ ，因此采用上文描述的方法组织 TLP 报文即可。

3. M 大于 $0x20$

如果 M 大于 $0x20$ ，此时 Capric 卡进行一次 DMA 写操作时，LogiCORE 需要向 RC 发送多个存储器写请求 TLP。我们假设 Capric 卡需要向 $[A_{31} A_{30} \cdots A_1 A_0 \sim B_{31} B_{30} \cdots B_1 B_0]$ 这段数据区域进行 DMA 写操作，而且这段数据区域的 M 大于 $0x20$ 。此时 DMA 写逻辑需要进行拆包操作。这个拆包操作需要遵循以下原则。

(1) TLP 传递的数据区域不能跨越 4 KB 边界

为此 Capric 卡首先需要分析 $A \sim B$ 这段是否跨越 4 KB 边界。值得注意的是，在 Capric 卡中，一次 DMA 写的长度小于 2048，因此其传递的数据区域至多会跨越一个 4 KB 边界。此时需要向 $A \sim \text{Tail}_{4096}(A)$ 和 $\text{Head}_{4096}(B) \sim B$ 这两段数据区域进行写操作，而且这两段数据区域的 M 都可能大于 $0x20$ 。下文采用的拆包方法可以保证在不进行 4 KB 边界检查的情况下，保证拆分后的 TLP 不会跨越 4 KB 边界。

(2) 尽量减少拆分后 TLP 的总个数

比如，可以将 $0x1000 \sim 10FF$ 这段数据区域拆分为 $0x1000 \sim 107F$ 和 $0x1080 \sim 0x10FF$ ，尽量利用 Max_Payload_Size 参数，而不使用更多的 TLP 进行数据传递。

(3) 拆分后的 TLP 尽量不跨越 Cache 行边界

虽然 PCIe 总线规范并没有规定拆分 TLP 的方法。但是将 $0x1000 \sim 10FF$ 这段数据区域拆分为 $0x1000 \sim 0x107E$ ， $0x107F \sim 0x108F$ 和 $0x1090 \sim 0x10FF$ ，从原理上讲是可行的，可是并不合理。

① 这段数据区域的有效长度小于等于 $0x80$ ，即 $M \leq 0x20$ 。

② 如果 $B > \text{Tail}_{4096}(A)$ 时，表示当前数据区域超越 4 KB 边界。

在 Capric 卡中，为了简化设计，当 M 大于 0x20 时将采用以下规则进行拆包处理。

- 第一个 TLP 的起始地址必须为 $0xA_{31}A_{30}\cdots A_2$ ，而其他 TLP 的起始地址必须为 0x80 字节对齐。
- 最后一个 TLP 的结束地址必须为 $0xB_{31}B_{30}\cdots B_2$ ，而其他 TLP 的结束地址必须为 0x80 字节对齐。

根据以上规则，我们可以将 $A_{31}A_{30}\cdots A_1A_0 \sim B_{31}B_{30}\cdots B_1B_0$ 这段数据区域划分为多个数据区域，其中每一个区域的 Length 字段不超过 0x20，而且每段数据区域以 128B 对齐。

$A_{31}A_{30}\cdots A_1A_0 \sim \text{Tail}_{128}(A)$
 $\text{Head}_{128}(A+128) \sim \text{Tail}_{128}(A+128)$
 ...
 $\text{Head}_{128}(A+n\times 128) \sim \text{Tail}_{128}(A+n\times 128)$
 ...
 $\text{Head}_{128}(B) \sim B_{31}B_{30}\cdots B_1B_0$

以上这些数据区域的 M 都小于或等于 0x20，而且都不会跨越 128B 边界，因此也不可能跨越 4 KB 边界。向这些数据区域传送数据时，TLP 各字段的设置参见 M 小于或等于 0x20 的情况。当 Capric 卡将这些存储器写请求 TLP 发送完毕后，可以向处理器提交中断请求。

12.2.2 DMA 读使用的 TLP

与 DMA 写模块相比，DMA 读模块的逻辑设计较为复杂。在 PCIe 总线中，存储器写 TLP 使用 Posted 总线传送方式，实现 DMA 写操作只需要使用存储器写 TLP 即可。而 PCIe 总线使用 Split 总线传送方式进行存储器读操作。因此一个 DMA 读过程由 EP 向 RC 发送“存储器读请求 TLP”，之后再由 RC 使用“存储器读完成 TLP”将数据传递给 EP。

当软件启动 DMA 读操作后，DMA 控制逻辑将根据需要读取数据区域的大小，决定发送存储器读请求 TLP 的个数，如果所读取数据区域的实际长度超过 Max_Read_Request_Size 参数[⊖]时，DMA 控制逻辑需要进行拆包处理，向 RC 发送多个存储器读请求 TLP，这些存储器读请求 TLP 将使用不同的 Tag。

当 RC 收到这些存储器读请求 TLP 后，将使用存储器读完成 TLP，将数据传递给 Capric 卡。其中一个存储器读请求 TLP（Tag 不同的报文）可能对应多个存储器读完成 TLP，而且这些存储器读完成 TLP 可以乱序到达。

在 Capric 卡的 DMA 读模块的设计中，首先需要进行拆包处理，其次需要合理地管理 Tag 资源，而最值得注意的是对存储器读完成 TLP 的乱序处理。为此在 Capric 卡中设置了一个单向循环链表 tag_queue，以便 Capric 卡发送存储器读请求 TLP，进行 Tag 资源的管理，并处理存储器读完成 TLP 的序。

1. tag_queue

Capric 卡的 DMA 读模块使用了一个单向循环队列 tag_queue，当然设计者也可以使用其他逻辑实现同样的功能。实际上对于 Capric 卡而言，设置这样的循环队列是奢侈的，因为

⊖ 在 LogiCORE 中该参数为 512B。

Capric 卡仅实现了基本的 DMA 读写操作。该循环队列实际上是为笔者的另一个设计，即 Cornus 卡^①准备的。

在 tag_queue 队列中，设置了头尾指针，分别为 tag_front 和 tag_rear，Capric 卡使用 8 位^②寄存器存放这两个指针。该队列的每一个 Entry 对应一个 tag 资源，其 Entry 号与 Tag 号一一对应。DMA 读模块从 tag_rear 指针获得当前可以使用的 tag 资源（相当于将获得的 tag 字段加入 tag_queue 中），并从 tag_front 指针处释放 tag 资源（相当于从 tag_queue 的头部释放资源），在 tag_front 和 tag_rear 之间的 Entry 保存正在使用的 tag 资源。tag_queue 队列的组成结构如图 12-3 所示。

在 Capric 卡复位时，tag_front 与 tag_rear 指针同时指向 Entry 0，此时 tag_queue 为空。当 Capric 卡需要使用 tag 资源时，首先判断 tag_queue 是否为满，如式 12-6 所示。

$$(\text{tag_rear}+1) \bmod 256 = \text{tag_front} \quad (12-6)$$

当 tag_queue 队列不满时，Capric 卡可以从 tag_queue 中获得 tag 资源，其值等于 tag_rear，然后将 tag_rear 更新为 $(\text{tag_rear}+1) \bmod 256$ （相当于将获得的 tag 加入到 tag_queue 队列中）。当 Capric 卡释放 tag 资源时，需要判断 tag_queue 是否为空，如式 12-7 所示。

$$\text{tag_rear} = \text{tag_front} \quad (12-7)$$

在 Capric 卡中，到达的存储器读完成 TLP 因为乱序的原因，其 tag 字段不一定与 tag_front 相等。Capric 卡错误处理逻辑需要判断到达的存储器读完成 TLP 的 tag 字段是否在 tag_front 和 tag_rear 之间（在图 12-3d 中，阴影部分的 Entry 在当前 tag_queue 中有效），其判断条件如式 12-8 所示。

$$(\text{tag}-\text{tag_front}) \bmod 256 < (\text{tag_rear}-\text{tag_front}) \bmod 256 \quad (12-8)$$

在 Cornus 卡中，tag_queue 队列的 Entry 由许多字段组成，在这些字段中“L”和“U”位对于 Capric 卡有意义。其中 U 为 Used 位，当该位为 1 时，表示对应 Entry 正在被使用，为 0 时表示没有被使用；而 L 为 Last 位，当该位为 1 时，表示对应 Entry 保存 DMA 操作最后一个存储器读请求 TLP。这些位的详细解释见下文。

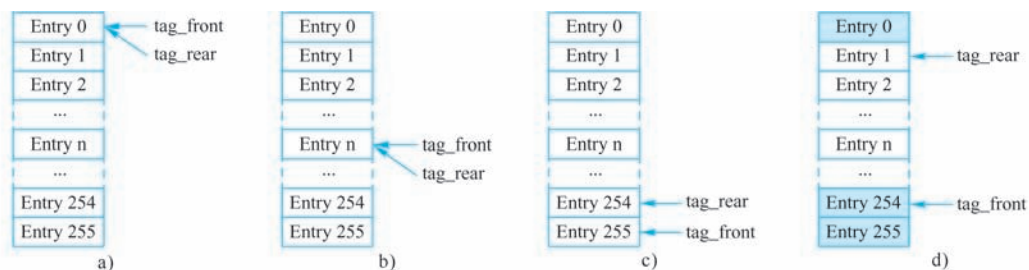


图 12-3 循环队列 tag_queue 的组成结构

a) tag_queue 的初始化 b) tag_queue 为空 c) tag_queue 为满 d) tag_queue 中的有效 Entry

2. Capric 卡发送存储器读请求 TLP

Capric 卡发送存储器读请求 TLP 与发送存储器写 TLP 的步骤较为类似，只是存储器读请求不含有 Data Payload，存储器读请求 TLP 的格式如图 6-8 所示。与存储器写请求 TLP 相

① Cornus 卡是一个基于 PCIe 总线的以太网卡，支持多通路 DMA 读写操作。

② 因为 tag_queue 队列的长度为 256。

比, 存储器读报文多了两个字段分别为 Requester ID 字段和 Tag 字段, 这两个字段合称为 Transaction ID 字段, 该字段的结构如图 6-9 所示。

从第 6.2.1 节中, 可以获知存储器读请求 TLP 使用地址路由方式, Capric 卡将存储器读请求 TLP 发送给 RC 时, 并不需要使用 Transaction ID 字段进行 ID 路由。但是存储器读完成 TLP 需要使用 ID 路由方式进行传送, 因而需要使用 Capric 卡的 Transaction ID 字段将存储器读完成 TLP 发送给 Capric 卡。

在 PCIe 总线中, 每一个数据传送都有唯一的 Transaction ID, Transaction ID 由 Requester ID 和 Tag 字段组成, 其中 Requester ID 由 HOST 处理器在系统初始化时设置。Capric 卡需要记录这个 Requester ID, 以便传送存储器读请求 TLP。

在存储器读请求 TLP 中其他字段的设置与存储器写请求 TLP 类似, 这些字段的设置参见上文。其中存储器读请求 TLP 的 Fmt 字段为 0b00/0b01, 表示使用 3DW/4DW 的 TLP 头, 而且不带数据; 而 Type 字段与存储器写请求 TLP 的 type 字段相等, 都为 0b0000。在 PCIe 总线中, 存储器写报文一定带有数据, 而存储器读请求一定不带数据, 这是 PCIe 总线区分存储器读请求 TLP 和存储器写 TLP 的方法。在设计中, Capric 卡使用 4DW 的读请求 TLP 头, 因为 Capric 卡内部使用 64b 数据总线, 使用 4DW 的报文头便于对界处理; 而存储器读完成报文只能使用 3DW 的报文头, 这为 Capric 卡的设计也带来了一些困难。

在存储器读请求 TLP 中, 需要重点处理的字段是 Tag。在 PCIe 总线中, 每个设备都有唯一的 Requester ID, 而且每一次数据传送使用不同的 Transaction ID, 在一次数据传送没有完成之前, 其他数据传送不能使用相同的 Transaction ID。在 PCIe 总线中, 使用 Tag 字段区分不同的 Transaction ID, 因为对于同一个 PCIe 设备发出的 TLP, Requester ID 字段都是相同的, 只有 Tag 字段不同。

当 Capric 卡向 RC 发送存储器读请求 TLP 时, 将从 tag_queue 中选择一个未用的 Tag 资源; 当 Capric 卡收齐与“存储器读请求 TLP”对应的“存储器读完成 TLP”[⊖]后, 将释放这个 Tag 资源, 之后其他存储器读请求 TLP 可以使用这个 Tag。

为此 Capric 卡需要使用一组数据缓冲维护这个 Tag 字段。在 PCIe 总线中, Tag 字段为 5 或者 8 位, 如果使能了 Phantom 功能, 一个 PCIe 设备可以使用更多的 Tag 资源, 详见第 4.3.2 节。在 Capric 卡中, 并没有使能 Phantom 功能, 而且使用的 Tag 字段为 8 位, 即 Device Capability 寄存器的 Extended Tag Field Supported 位为 1, 该寄存器的详细描述见第 4.3.2 节。

Capric 卡使用 tag_rear 指针从 tag_queue 队列中获得未用的 tag 资源, 并设置 tag_queue 中对应 Entry 的 L 和 U 位。Capric 卡发送存储器读请求 TLP 时, 首先需要判断本次 DMA 读操作一共需要向 RC 发送几个存储器读请求 TLP。在 Capric 卡中, 一次 DMA 读可以使用的最大传送单位为 2047B, 该值超过 Capric 卡的 Max_Read_Request_Size 参数 (512B), 因此 Capric 卡发送存储器读请求 TLP 时需要进行拆包操作。

如果 Capric 卡读取 $A(A_{31}A_{30}\cdots A_1A_0)\sim B(B_{31}B_{30}\cdots B_1B_0)$ 这段数据区域时, 需要首先计算每段数据区域的实际长度 M, 该长度的计算与公式 12-4 相同。如果 M 小于或等于 512B, 需要继续检查这段数据区域是否超过 4 KB 边界, 如果超过需要将这段数据区域分为 A ~

⊖ 一个存储器读请求 TLP 可能对应若干个存储器读完成 TLP。

Tail₄₀₉₆(A) 和 Head₄₀₉₆(B) ~ B 这两段数据区域进行读操作。

如果 M 大于 512B, Capric 卡需要进行拆包处理, 将 A~B 这段数据区域分割为若干个数据区域, 其中每一段数据区域的 Length 字段不超过 0x80, 而且为 512B 对齐。Capric 卡使用的拆包方法如下所示。

第 1 个存储器读请求 TLP 对应的数据区域: $A_{31}A_{30}\cdots A_1A_0 \sim \text{Tail}_{512}(A)$

第 2 个存储器读请求 TLP 对应的数据区域: $\text{Head}_{512}(A+512) \sim \text{Tail}_{512}(A+512)$

...

第 n 个存储器读请求 TLP 对应的数据区域: $\text{Head}_{512}(A+n\times 512) \sim \text{Tail}_{512}(A+n\times 512)$

...

最后 1 个存储器读请求 TLP 对应的数据区域: $\text{Head}_{512}(B) \sim B_{31}B_{30}\cdots B_1B_0$

以上这些数据区域的 M 都小于 0x80, 而且都不会跨越 512B 边界, 因此也不可能超过 4 KB 边界。使用以上拆包方法, Capric 卡可以获得若干个 M 小于或等于 0x80 的数据区域, 因此 Capric 卡可以使用一个存储器读请求从主存储器获得以上每段数据区域对应的数据。Capric 卡使用 4DW 的 TLP 头, 其格式如图 12-4 所示。

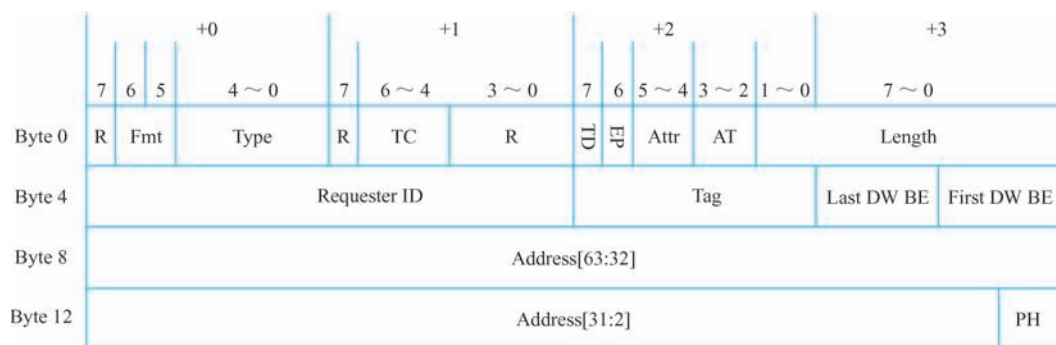


图 12-4 存储器读请求 TLP 头格式

Capric 卡向 RC 发送这些存储器读请求 TLP 的详细步骤如下。

1) 组织存储器读请求 TLP, 其中 Byte 0 中的字段、First/Last DW BE 字段和 Address 字段与存储器写请求 TLP 的对应字段相同, 本小节对此不做详细描述。而 Requester ID 字段由 Host 处理器在初始化时设置。

2) Capric 卡从 tag_queue 队列中获得 tag 字段, 首先通过公式 12-6 判断 tag_queue 队列是否有可用 tag 资源, 如果没有则循环等待公式 12-6 成立; 如果有可用 tag, 则继续。在 Capric 卡中, Address[63:2] 较小的存储器读请求使用的 Tag 字段也较小。

3) 当前存储器读请求使用 tag_rear 作为 tag 字段, 同时置 tag_queue[tag_rear].U 为 1, 表示当前 Entry 已被使用。

4) 在一次 DMA 读操作时, Capric 卡可能需要进行拆包操作。如果当前存储器读请求 TLP 是最后一个 TLP, 则将 tag_queue[tag_rear].L 位置为 1, 否则置为 0。

5) 将 tag_rear 赋值为 (tag_rear+1) mod 256, 然后发送该存储器读请求 TLP。如果 L 位为 0 时转 1), 表示与当前 DMA 操作对应的存储器读请求 TLP 还没有发送完毕; 否则结束存储器读请求报文的发送。

Capric 卡发送存储器读请求 TLP 时, 还需要考虑一个细节问题。在 PCIe 总线中, EP 为

CplH 和 CplD 提供的 Credit 为 0，即 Infinite Credit，详见第 9.3.2 节。这意味着 EP 每发送一个存储器读请求，必须为对应的存储器读完成的报文头和数据预留缓冲。

假设 Capric 卡连续向 RC 发送了 256 个存储器读请求 TLP，其中每个存储器读请求 TLP 访问的数据区域为 512 B，而在 RC 发送的存储器读完成 TLP 中一次只能携带 64 B。此时即便不考虑对界的问题，Capric 卡也需要为存储器读完成预留较大的缓冲空间，该空间由两部分组成，如下所示。

- 1) 预留存储器读完成 TLP 头的空间大小为 8192 B ($256 \times 512 \times 4 / 64$)。
- 2) 预留存储器读完成 TLP 数据的空间大小 128 KB (256×512 B)。

在硬件设计中，为了提高 DMA 读的数据传送效率，还可以使能 Phantom 功能，此时 EP 能够发送的存储器读请求 TLP 更多。如果 EP 经过若干级 Switch 后，才能到达 RC，此时 RC 可能正在处理其他 EP 的存储器读请求而不会立即处理这些存储器读请求，此时该 EP 可能长时间不能收到存储器读完成 TLP，从而无法释放预留的数据缓冲。因此 EP 可能会因为没有数据缓冲，而无法继续发送存储器读请求 TLP。

实际上，硬件为存储器读完成报文预留的数据缓冲是有限的，一般不会预留 136 KB 大小的空间，在 LogiCORE 中，为 CplH 预留的缓冲单元为 33~36 个，而为 CplD 中的数据预留的缓冲为 2176~2304 B。

由此可以发现如果 RC 没有及时地将存储器读完成 TLP 发送回来，Capric 卡最多在连续发送 33 个存储器读请求后（假设存储器读请求使用 4DW 的报文头），就因为无法为存储器读完成的报文头提供足够的缓冲而不能继续发送；此外如果每个存储器读请求所访问的数据区域都是 512 B，Capric 卡最多在连续发送 4 个这样存储器读请求后，就不能继续发送这样的存储器读请求，从而造成发送流水线的中断。

由此可以发现，在 LogiCORE 中，由于预留的缓冲有限，Capric 卡在使用 PCIe 总线要求的 Infinite Credit 机制时，将因为预留缓冲不足而造成流水线的中断。为此 LogiCORE 提供了三种流量控制机制，这三种流量控制机制在重构 LogiCORE 时选择使用，系统软件不能通过修改寄存器动态配置这些流控方式。

(1) Infinite Credit

该方式与 PCIe 总线规范兼容。如果 Capric 卡使用 Infinite Credit 方式，当 LogiCORE 内部的接收缓冲不足时，Capric 卡不能向 RC 发送存储器读请求报文。根据上文的讨论，由于 LogiCORE 内部的接收缓冲不足，因此使用该方式在某种程度上，将造成 DMA 读流水线的中断，从而影响 DMA 读的效率。

(2) One Posted/Non-Posted Header

该方式与 PCIe 总线规范不兼容，使用这种方式时，EP 每次为上游端口发送的 Posted 请求和 Non-Posted 请求提供最小的 Credit，相当于 EP 每一次只能发送一个存储器读请求 TLP，而得到与之对应的存储器读完成 TLP 后，再提交下一个存储器读请求 TLP。使用这种方法将严重影响 DMA 读的效率。LogiCORE 提供的这种方法可能是用于调试目的。在正常情况下设计者不应该使用这种方式。

(3) Non-Infinite Credit

该方式与 PCIe 总线规范不兼容，使用这种方式时，EP 并没有给上游端口提供无限量的 Credit，而是根据预留接收缓冲的实际使用情况，为上游端口提供 Credit。Capric 卡采用这种

方式，发送存储器读请求 TLP 时，并不会为存储器读完成 TLP 事先预留接收缓冲，从而在发送存储器读请求 TLP 时，并不会因为接收缓冲不足而被中断，因此提高了 Capric 卡发送存储器读请求 TLP 的效率。

LogiCORE 在接收存储器读完成报文时，将根据预留缓冲的实际大小为对端提供 Credit。虽然采用这种方法与 PCIe 总线规范要求的 Infinite Credit 并不兼容。但是使用这种方法避免了在发送存储器读请求时，因为接收缓冲不足而引发的流水线中断，从而 Capric 卡可以连续发送多个存储器读请求，无论是否具有足够的接收缓冲。

而 Capric 卡将以较快的速度从 LogiCORE 的预留缓冲中获得数据，因此在多数时间里，不会因为预留缓冲被对端设备耗尽而引发接收流水线的中断。因此在实际设计中，Capric 卡使用了这种流量控制方式。

3. Capric 卡接收存储器读完成 TLP

在 PCIe 总线中，EP 发出的存储器读请求可以超越之前的存储器读请求，而且当存储器完成报文使用的 Transaction ID 不同时，存储器读完成 TLP 也可能超越之前的存储器读完成 TLP，这将造成存储器读完成 TLP 乱序到达 Capric 卡。

Capric 卡必须注意处理这个乱序问题。下面举例说明这个序的问题，假设 Capric 卡向处理器的 0x1000~0x11FF 这段数据区域发送存储器读请求 TLP，RC 将通过存储器读完成 TLP 向 Capric 卡传递数据。如果这个 RC 使用的 RCB 为 64B，则 RC 可以使用 4 个存储器读完成 TLP 发送这些数据，如图 12-5 所示。

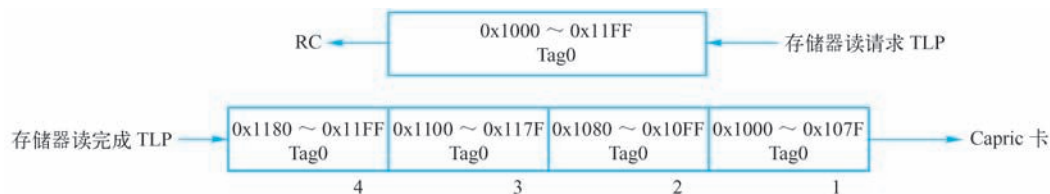


图 12-5 使用一个存储器读 TLP 对 0x1000~0x11FF 进行 DMA 读操作

其中第 1 个存储器读完成 TLP 的数据来自 0x1000~0x107F 这段数据区域；第 2 个存储器读完成 TLP 的数据来自 0x1080~0x10FF 这段数据区域；第 3 个存储器读完成 TLP 的数据来自 0x1100~0x117F 这段数据区域；而第 4 个存储器读完成 TLP 的数据来自 0x1180~0x11FF 这段数据区域。在这种情况下，存储器读完成报文将按序到达 Capric 卡，从而并不会对 Capric 卡的硬件逻辑造成影响。

如果 Capric 卡使用 2 个存储器读请求 TLP 向处理器的 0x1000~0x11FF 这段数据区域发起存储器读请求。其中第 1 个存储器读请求 TLP (tag 0) 向处理器的 0x1000~0x10FF 这段数据区域发起存储器读请求，而第 2 个存储器读请求 TLP (tag 1) 向处理器的 0x1100~0x11FF 这段数据区域发起存储器读请求。此时来自 RC 的存储器读完成报文可能乱序到达 Capric 卡，如图 12-6 所示。

此时 RC 依然使用 4 个存储器读完成 TLP 向 Capric 卡发送这些数据，但是由于序的问题，这 4 个存储器读完成 TLP 可能以 2 种不同的顺序发向 Capric 卡。当然 RC 还可以以其他顺序向 Capric 卡发送这些 TLP，本节并不列出所有可能的顺序。

(1) 第 1 种序

- 第 1 个存储器读完成 TLP 的数据来自 0x1000~0x107F 这段数据区域 (Tag 0)。



图 12-6 使用两个存储器读 TLP 对 0x1000~0x11FF 进行 DMA 读操作

- 第 2 个存储器读完成 TLP 的数据来自 0x1080~0x10FF 这段数据区域 (Tag 0)。
- 第 3 个存储器读完成 TLP 的数据来自 0x1100~0x117F 这段数据区域 (Tag 1)。
- 第 4 个存储器读完成 TLP 的数据来自 0x1180~0x11FF 这段数据区域 (Tag 1)。

(2) 第 2 种序

- 第 1 个存储器读完成 TLP 的数据来自 0x1100~0x117F 这段数据区域 (Tag 1)。
- 第 2 个存储器读完成 TLP 的数据来自 0x1000~0x107F 这段数据区域 (Tag 0)。
- 第 3 个存储器读完成 TLP 的数据来自 0x1180~0x11FF 这段数据区域 (Tag 1)。
- 第 4 个存储器读完成 TLP 的数据来自 0x1080~0x10FF 这段数据区域 (Tag 0)。

这个乱序问题为 Capric 卡的 DMA 读机制的设计带来了不小的麻烦。因为在“第 2 种序”的情况下，先发出去的存储器读请求 TLP，后接收到与之对应的存储器完成报文。不过值得庆幸的是，对于一个存储器读请求 TLP，其对应的存储器完成报文虽然也有多个，但是这些报文将以地址顺序先后到达。如向 0x1000~0x10FF 这段数据区域发送的存储器读请求，其存储器完成报文虽然被分解为两个，但一定是传送 0x1000~0x107F 这段区域的存储器读完成 TLP 率先到达，而传送 0x1080~0x10FF 这段区域的存储器读完成 TLP 随后到达。

在 Capric 卡的设计中必须考虑这个乱序问题，因为 Capric 卡进行 DMA 读操作时，所读取的数据区域可能超过 Max_Read_Request_Size 参数，此时 Capric 卡对这段数据区域进行 DMA 读时，必须向 RC 发出多个存储器读请求 TLP，参见上文。

与 Capric 卡发送存储器读请求 TLP 相比，Capric 卡处理存储器读完成 TLP 的过程更为复杂。当 Capric 卡收到来自 RC 的存储器完成报文后，需要进行一系列检查。存储器读完成 TLP 的格式如图 12-7 所示。

Capric 卡接收到存储器读完成 TLP 后，首先需要检查报文头。其中 Fmt 字段必须为 0b010，Type 字段必须为 0b01010。除此之外 Capric 卡还需要进行以下检查。

1) 存储器读完成 TLP 的 Requester ID 字段必须与 Capric 卡的 Requester ID 字段相等。否则该存储器读完成 TLP 被认为是“Unexpected Completion”报文，Capric 卡需要丢弃该存储器读完成 TLP，并将 ERR 寄存器的 UC 位置 1。

2) 检查存储器读完成 TLP 的 Status 字段，如果 Status 字段不为 0b000，则表示接收到的 TLP 出现错误。如果 Status 字段为 0b001 或者 0b100 时，Capric 卡需要丢弃该存储器读完成 TLP，并将 ERR 寄存器的相应位置 1。

3) 检查存储器读完成 TLP 的 tag 字段，确认当前报文是否与已经发出的存储器读请求

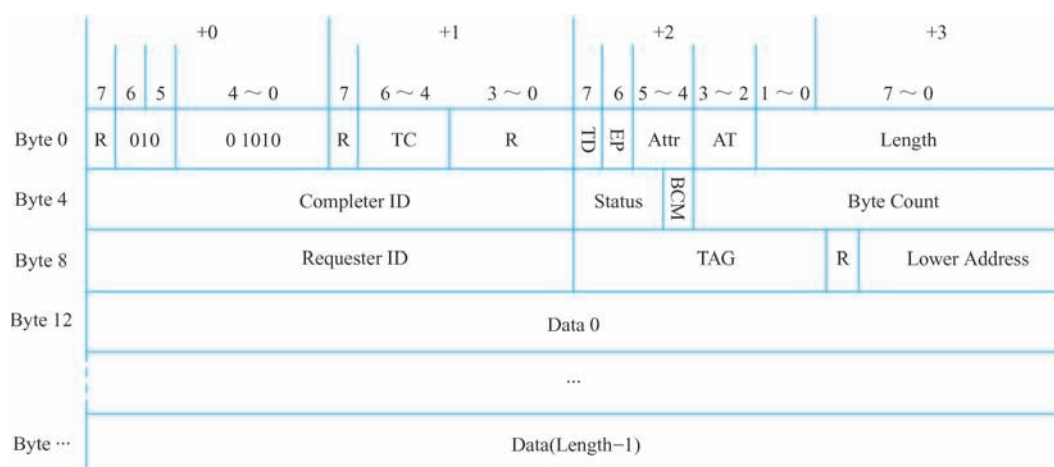


图 12-7 存储器读完成 TLP

TLP 对应，检查方法如公式 12-8 所示。

4) 此外 Capric 卡还需要检查 EP 位，TD 位、TC 字段和 Attr 字段。

Capric 卡的接收部件成功完成这些检查之后，将从存储器读完成 TLP 中获取数据。PCIe 总线规定，一个存储器读请求 TLP，可以对应多个存储器读完成 TLP。这为 Capric 卡的设计带来了一定的困难，为此 Capric 卡需要将存储器读完成 TLP 全部收齐后，才能释放相应的 tag 资源，最后将 tag_queue 对应 Entry 的 U 位和 L 位清零。

存储器读完成报文虽然可能有多个，但是这些报文将以地址顺序先后到达。因此 Capric 卡首先需要分析 tag 字段，从而确定当前存储器读完成 TLP 与哪个存储器读请求 TLP 对应。其中第 1 个存储器读完成 TLP 与存储器读请求 TLP 起始地址对应，之后的存储器读完成 TLP 将地址顺序依次到达。假定向 $[A_{31}A_{30} \cdots A_1A_0 \sim B_{31}B_{30} \cdots B_1B_0]$ 这段数据区域发起存储器读请求时，RC 将发送多个存储器读完成 TLP，并以下列顺序到达。

$$\begin{aligned}
 & [A_{31}A_{30} \cdots A_1A_0 \sim \text{Tail}_{64}(A_{31}A_{30} \cdots A_1A_0)] \\
 & [\text{Head}_{64}(A_{31}A_{30} \cdots A_1A_0 + 64) \sim \text{Tail}_{64}(A_{31}A_{30} \cdots A_1A_0 + 64)] \\
 & \dots \\
 & [\text{Head}_{64}(A_{31}A_{30} \cdots A_1A_0 + n * 64) \sim \text{Tail}_{64}(A_{31}A_{30} \cdots A_1A_0 + n * 64)] \\
 & \dots \\
 & [\text{Head}_{64}(B_{31}B_{30} \cdots B_1B_0) \sim B_{31}B_{30} \cdots B_1B_0]
 \end{aligned}$$

当然 RC 也可能向 Capric 发送一个存储器读完成 TLP，传递 $[A_{31}A_{30} \cdots A_1A_0 \sim B_{31}B_{30} \cdots B_1B_0]$ 数据区域中的所有数据。无论这些存储器读完成 TLP 以什么样的形式到达，Capric 卡都需要正确接收这个存储器读完成 TLP。

Capric 卡首先分析存储器读完成 TLP 的 Length 字段，在该字段中存放当前存储器读完成 TLP 的长度，值得注意的是 Length 字段所存放的长度，可能超过这个存储器完成报文的包含的有效数据长度，因为地址 $A_{31}A_{30} \cdots A_1A_0$ 很可能不是 1DW 对界，而 Length 字段存放的最小数据单位为 1DW。此时 Capric 卡必须正确识别存储器读完成 TLP 中 Data0 (即第一个双字) 中包含的有效数据，以及 Data(Length-1) (即最后一个双字) 中包含的有效数据。

在 RC 发送给 Capric 卡的多个存储器读完成 TLP 中，只有第 1 个存储器读完成 TLP 所对应的存储器区域的起始地址可能不是 DW 对界；而如果存在其他存储器读完成 TLP，那么这些报文所对应存储器区域的起始地址至少是 64B 对界的，也可能是 128B 对界的^①。

但是存储器读完成 TLP 并不含有 First DW BE 字段，此时 Capric 卡需要使用存储器读完成 TLP 中的 Lower Address 字段识别 Data0 中的有效字节。

在第 1 个存储器读完成 TLP 中，Lower Address[1:0] = A_1A_0 ，对于其他存储器读完成 TLP，其 Low Address[1:0] = 0b00。因此通过 Lower Address 字段，可以识别 Data0 中第一个有效数据，即 Data0[A_1A_0]为第一个有效数据。

存储器读完成 TLP 并没有设置 Last DW BE 字段，Capric 卡需要使用 Byte Count 和 Lower Address 字段联合识别 Data(Length-1)中的有效数据。如果当前存储器读完成 TLP 不是最后一个 TLP，那么其 Data(Length-1)中的数据全部有效。因为 PCIe 总线规定，如果 RC 为 1 个存储器读请求 TLP 发送多个存储器读完成 TLP，如果这个存储器读完成 TLP 不是最后一个报文，那么其结束地址必须 64B 对界。

如果当前存储器读完成 TLP 不是第 1 个 TLP，那么其 Lower Address[1:0] = 0b00。在这两种情况下，Data(Length-1)中的有效数据较易计算。但是有一个特例情况，就是 RC 只发出了一个存储器读完成 TLP 给 Capric 卡，此时这个 TLP 既是第一个存储器读完成 TLP 也是最后一个存储器读完成 TLP。但是无论是上述哪种方式，依然存在计算 Data(Length-1)中的有效数据的通用方法，如式 12-9 所示。

$$0bX_1X_0 = \text{LowAddress}[1:0] + \text{ByteCount}[1:0] - 0b01 \quad (12-9)$$

其中 Data(Length-1)[X_1X_0]为存储器读完成 TLP 中最后一个有效数据。Capric 卡计算完毕存储器读完成 TLP 的 Data0 和 Data(Length-1)中的有效数据后，还需要判断当前存储器读完成 TLP 是不是 RC 发出的最后一个与当前 tag 对应的存储器读完成 TLP。为直观起见，以图 12-8 为例说明如何计算当前存储器读完成 TLP 是否为最后一个报文。

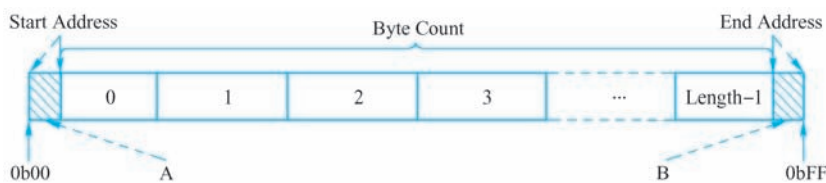


图 12-8 最后一个存储器读完成 TLP 的判断方法

如上图所示，Start Address 为存储器读完成 TLP 的起始地址，而 End Address 为存储器读完成 TLP 的结束地址。在一个存储器读完成 TLP 中，我们无法得到 Start Address 和 End Address 的确切的数值，因为存储器读完成 TLP 不包含 Address 字段，但是可以得到阴影 A 和阴影 B 的大小。其中阴影 A 的大小为 Low Address[1:0]，而阴影 B 的大小为 $0b11 - 0bX_1X_0$ 。

如果当前 TLP 的 Byte Count 字段加上阴影 A 和 B 的大小与 $\text{Length} \times 4$ 相等，即公式 12-10 成立时，该 TLP 为 RC 发给 Capric 卡的最后一个存储器读完成 TLP。

$$(\text{Byte Count} + \text{Low Address}[1:0] + 0b11 - 0bX_1X_0) = \text{Length} \ll 2 \quad (12-10)$$

① 采用哪种对界方式与 RCB 参数相关，不同处理器使用的 RCB 参数并不相同。

请读者重新阅读第 6.3.2 节，深入理解 Byte Count 参数的含义，以加深对公式 12-10 的理解。在 Capric 卡的硬件设计中，需要使用该公式识别最后一个到达的存储器读完成 TLP。

在 Capric 卡接收到最后一个存储器完成 TLP 之后，将完成一次存储器读请求。当最后一个存储器读请求完成后，将完成一次 DMA 读操作。Capric 卡接收存储器读完成 TLP 的详细步骤如下所示。

1) 首先进行报文检查。如果通过这些检查后，将从存储器读完成报文中获得数据填入相应 SRAM 的对应区域中。

2) DMA 读逻辑通过存储器读完成 TLP 的 tag 字段在 tag_queue 中查找对应的 Entry。如果当前存储器读完成是最后一个 TLP，将该 Entry 的 U 位清零。此时如果该 Entry 的 L 位为 1，表示本次 DMA 读结束，并向处理器提交中断请求，同时清除 L 位，并置相应的中断状态寄存器。Cormus 卡支持多路并发的 DMA 读操作，因此需要在 Entry 中设置 L 位。

3) DMA 读模块可能会更新 tag_front 指针，如果 tag 字段不等于 tag_front 指针，读模块不能更新 tag_front，而仅是将对应 Entry 的 U 位清零；如果相同则将 tag_front 更新为 $(\text{tag_front} + 1) \bmod 256$ ，同时将 U 位清零。

4) 之后 DMA 读模块继续判断 tag_queue[tag_front] 的 U 位是否为 0。如果该位为 0，将 tag_front 更新为 $(\text{tag_front} + 1) \bmod 256$ ，然后继续判断 tag_queue[tag_front] 的 U 位是否为 0，直到公式 12-7 成立，或者 tag_queue[tag_front] 的 U 位为 1。

12.2.3 Capric 卡的中断请求

Capric 卡支持两种中断请求方式，一种是 Legacy INTx 方式，另一种是 MSI 中断方式。Capric 卡需要向 RC 发送两个 Legacy INTx 中断消息，一个是 Assert INTx，另一个是 Deassert INTx，以实现 Legacy INTx 中断方式。第 6.3.4 节详细介绍了这种中断请求方式。这种中断请求方式虽然使用了 INTx 消息，但是其原理与电平触发方式类似，而 MSI 中断方式的工作原理与边沿触发方式类似。因此系统软件对 Capric 卡的这两种中断请求的处理并不相同。

在第 10.3 节中，我们曾详细讨论了电平触发与边沿触发的区别。其中采用电平触发不会丢失中断请求，而采用边沿触发将会丢失中断请求。Capric 卡可以保证即便使用了 MSI 中断机制，也不会丢失中断请求。

MSI 中断机制使用存储器写 TLP 实现，这个存储器写 TLP 的目的地址为 MSI Capability 结构中的 Message Address 字段，而数据为 Message Data 寄存器中的值。Message Address 字段和 Message Data 字段由系统软件在初始化时填写。在不同的处理器体系结构中，系统软件填写的这两个字段的数据并不相同，详见第 10.2 节和第 10.3 节。

LogiCORE 内部实现了 MSI 中断机制，Capric 卡仅需一些简单的组合逻辑即可实现 MSI 中断机制。Capric 卡需要根据 INT_REG 寄存器的信息，决定如何发送中断请求，这部分中断逻辑的实现较为简单，本节对此不做进一步说明。

12.3 基于 PCIe 总线的设备驱动

本节简要介绍 Capric 卡在 Linux 系统中使用的 Char 类型设备驱动程序。为便于读者理解 Linux 系统 PCIe 总线驱动程序的实现构架，本节将详细介绍 Capric 卡的初始化、DMA 读、

DMA 写、中断服务例程和关闭过程，但是并不会过多介绍和 Linux 系统相关的知识，而是重点介绍系统软件如何管理和配置 PCIe 设备。

12.3.1 Capric 卡驱动程序的加载与卸载

在 Linux 系统中，Capric 卡驱动程序的加载与卸载的过程如源代码 12-1 所示。这部分程序并不会直接操作 PCIe 设备，而是通过 `pci_register_driver` 函数向内核注册一个 `pci_driver` 结构，即 `capric_drv`，并由 `capric_probe` 函数完成 Capric 卡的初始化。

源代码 12-1 Capric 卡驱动程序的加载与卸载

```
static struct pci_device_id capric_ids[] = {
    { PCI_DEVICE(PCI_VENDOR_ID_XILINX, PCI_DEVICE_ID_EP_PIPE), },
    { 0, }
};
...
static struct pci_driver capric_drv = {
    .name = DEV_NAME,
    .id_table = capric_ids,
    .probe = capric_probe,
    .remove = capric_remove,
};

static int __init capric_init(void)
{
    int result;

    result = pci_register_driver(&capric_drv);
    return result;
}

static void capric_exit(void)
{
    pci_unregister_driver(&capric_drv);
}

module_init(capric_init);
module_exit(capric_exit);
```

在上述源代码中，`pci_register_driver` 函数的主要作用是将 `capric_drv` 结构与 PCI 设备的 `pci_dev` 结构^①进行绑定，并在初始化时执行 `capric_probe` 函数，而在结束时执行 `capric_`

① 这些 `pci_dev` 结构在 Linux 系统对 PCI 总线枚举时建立。在加载 Capric 卡驱动程序之前，这些 `pci_dev` 结构已经存在。Linux 系统对 PCI 总线的枚举过程见第 14.3 节。

remove 函数。这段源代码的主要作用是将 Capric 卡驱动程序使用的“软件结构 pci_driver”与“硬件结构 pci_dev”建立联系。本文并不会深入分析 pci_register_driver 和 pci_unregister_driver 函数的实现细节，而仅介绍该函数的执行顺序。对 Linux 系统有一定经验的读者，可以从中获得必要的知识。

pci_register_driver 函数首先调用 _pci_register_driver→driver_register→bus_add_driver 函数。bus_add_driver 函数进行一些必要的初始化操作后，调用 driver_attach→bus_for_each_dev 函数查找 Capric 卡的 pci_dev 结构。

在 Linux 系统中，bus_for_each_dev 函数是一个重要的函数，该函数将遍历 Capric 卡所在 PCI 总线树上的所有 pci_dev 结构，并依次判断 pci_dev 结构中的 Device ID、Vendor ID 等信息是否与 capric_ids 结构中对应的信息相同，如果相同则调用 capric_probe 函数。bus_for_each_dev 函数调用 _driver_attach 函数实现该过程。

_driver_attach 函数调用 drv→bus→match 函数（即 pci_bus_match 函数），而 pci_bus_match 函数将继续调用 pci_match_device→pci_match_id 函数，判断 capric_ids 所包含的内容是否在当前 PCI 总线树的 pci_dev 中出现。如果出现，将 capric_drv 结构与实际的 PCI 设备进行绑定。之后继续调用 driver_probe_device→really_probe 函数。

really_probe 函数将调用 dev→bus→probe 函数（即 pci_device_probe 函数），pci_device_probe 函数将调用 _pci_device_probe→pci_call_probe→local_pci_probe 函数，并最终调用 Capric 卡的 probe 函数，即 capric_probe 函数。

Capric 卡的卸载过程是加载的逆过程，其调用顺序为 pci_unregister_driver 函数、driver_unregister 函数、bus_remove_driver 函数、driver_detach 函数和 _device_release_driver 函数，并最终调用 capric_remove 函数。

对于 Capric 卡，初始化与结束操作是在 capric_probe 和 capric_remove 函数中完成的。在 capric_ids 结构中使用的 id 号，是联系 Capric 卡的 pci_driver 结构和 pci_dev 结构的桥梁。在该结构中的 PCI_VENDOR_ID_XILINX 和 PCI_DEVICE_ID_EP_PIPE 即为 Capric 卡的 Vendor ID 和 Device ID，分别为 0x10EE 和 0x0007。

12.3.2 Capric 卡的初始化与关闭

Capric 卡的 probe 函数完成硬件初始化和一些 Linux 系统相关的初始化操作。当 Linux PCI 在当前 PCI 总线树中，发现 Capric 卡后，由 local_pci_probe 调用 capric_probe 函数，该函数具有两个入口参数 pci_dev 和 ids，其执行过程如源代码 12-2~12-5 所示。

源代码 12-2 Capric 卡的硬件初始化片段 1

```
static struct capric_private * adapter;
...
static int capric_probe(struct pci_dev * pci_dev,
                      const struct pci_device_id * ids)
{
    int result;
    resource_size_t base_addr;
```

```

        unsigned long length;
    ...

    adapter = kmalloc( sizeof( struct capric_private ), GFP_KERNEL );

    if( unlikely( ! adapter ) )
        return -ENOMEM;
    adapter->pci_dev = pci_dev;
    ...

    result = pci_enable_device( pci_dev );

    if( unlikely( result ) )
        goto free_adapter;

```

首先 `capric_probe` 函数从 `local_pci_probe` 函数中获得 Capric 卡对应的 `pci_dev` 描述符，在 Linux 系统中每一个 PCI/PCIe 设备都与唯一的 `pci_dev` 描述符对应。`pci_dev` 描述符包含 PCIe 设备的全部信息，该结构较为简单，在 `./include/linux/pci.h` 文件中定义，本节并不对该结构进行详细介绍。

这段函数首先为全局指针 `adapter` 分配空间，在全局指针 `adapter` 中记录了一些 Capric 卡需要使用的私有参数，包括 Capric 卡使用的 `pci_dev`。这段程序为 `adapter` 分配完内存空间后，将调用 `pci_enable_device` 函数使能 PCIe 设备。`pci_enable_device` 函数的主要作用是修改 Capric 卡 PCI 配置空间 Command 寄存器的 I/O Space 位和 Memory Space 位。

`pci_enable_device` 函数最终调用 `pci_enable_resources` 函数，并由 `pci_enable_resources` 函数扫描 Capric 卡的 BAR0~5 空间，如果这些 BAR0~5 空间用到了 I/O 或者 Memory 空间，则将 I/O Space 位和 Memory Space 位置 1。`pci_enable_device` 函数最后调用 `pcibios_enable_irq` 函数分配 PCI 设备使用的中断向量号^①。此后处理器可以使用存储器或者 I/O 指令与 Capric 卡通信。`pci_enable_device` 函数还有一个用途是置 PCI 设备的 D-State 为 D0。

Linux PowerPC^② 与 Linux x86 在此处的处理基本类似。只是在 PowerPC 处理器系统中，某些 PCI 设备支持存储器写并无效周期，此时 `pci_enable_device` 函数还需要使能 PCI 设备的 Memory Write and Invalidate 位，同时需要填写配置空间的 Cache Line Size 寄存器^③。

Linux x86 的 MSI 中断机制处理过程与 Linux PowerPC 也不尽相同。在 Linux x86 中，一个 PCIe 设备使能或者不使能 MSI 中断机制时，其 `pci_dev→irq` 参数并不相同。

如果其他设备驱动程序再次调用 `pci_enable_device` 函数使能该设备时，该函数仅增加 `pci_dev` 的引用计数，并不会重新使能该 PCI 设备。与此对应 `pci_disable_device` 函数只有在 `pci_dev` 的引用计数为 0 之后，才能关闭 `pci_dev` 结构。当两个以上的设备驱动程序操纵相同的硬件时，会出现这种情况。

① 如果该 PCI 设备没有使用 MSI 或者 MSI-X 机制时，才进行这种操作。

② 本书分别用 Linux PowerPC 和 Linux x86 代表基于 PowerQUICC（32 位）和 x86 处理器的 Linux 系统。

③ 该步骤如 `pmac_pci_enable_device_hook` 函数所示。

源代码 12-3 Capric 卡的硬件初始化片段 2

```

pci_set_master(pci_dev);
// pci_try_set_mwi(pci_dev);

result = pci_set_dma_mask(pci_dev, DMA_MASK);

if(unlikely(result)) {
    PDEBUG("can not set dma mask ... \n");
    goto disable_pci_dev;
}

```

pci_set_master 函数将 Capric 卡 PCI 配置空间 Command 寄存器的 Bus Master 位置 1，表示 Capric 卡可以作为 PCI 总线的主设备。Capric 卡是基于 PCIe 总线的设备，而 PCIe 总线不支持存储器写并无效操作，因此这段程序不需要使用 pci_try_set_mwi 函数设置 Command 寄存器的 Memory Write and Invalidate 位。

pci_set_dma_mask 函数设置 PCIe 设备使用的 DMA 掩码。Capric 卡对一段内存进行 DMA 操作时，需要使用这段内存存在 PCI 总线域的物理地址 pci_address，如果这段内存存在存储器域的物理地址 physical_address & DMA_MASK = physical_address 时，表示 Capric 卡可以对这段内存进行 DMA 操作。

x86 处理器可以使用 pci_dma_supported 函数获得最合适的 DMA_MASK 参数；而在 PowerPC 处理器中允许 PCIe 设备访问的主存储器地址范围在 Inbound 寄存器组中定义，只有在 Inbound 寄存器窗口中映射的物理地址才能被 PCIe 设备访问，有关 Inbound 寄存器组的详细说明见第 2.2 节。在 x86 和 PowerPC 处理器中，pci_set_dma_mask 函数的实现方法不同。

源代码 12-4 Capric 卡的硬件初始化片段 3

```

result = pci_request_regions(pci_dev, DEV_NAME);
if(unlikely(result))
    goto disable_pci_dev;

base_addr = pci_resource_start(pci_dev, 0);

if(unlikely(! base_addr)) {
    result = -EIO;
    PDEBUG("no MMIO ... \n");
    goto release_regions;
}

if(unlikely((length = pci_resource_len(pci_dev, 0) < BAR0_BYTE_SIZE))) {
    result = -EIO;
    PDEBUG("MMIO is too small ... \n");
    goto release_regions;
}

```

```

    }

    adapter->pci_bar0 = ioremap(base_addr, length);

    if(unlikely(! adapter->pci_bar0)) {
        result = -EIO;
        PDEBUG(" cannot map MMIO... \n");
        goto release_regions;
    }

```

这段源代码调用 `pci_request_regions` 函数使 `DEV_NAME` 对应的驱动程序成为 `pci_dev` 存储器资源的拥有者。在 Linux 系统中所有存储器映射的寄存器和 I/O 映射的寄存器都使用 `ioresources` 进行管理。每一组存储器空间都对应一个 `resource` 结构，`pci_request_regions` 函数经过一系列函数调用，最终调用 `request_region` 函数，将 Capric 卡的 BAR0 空间使用的 `resource` 结构，其 `name` 参数设置为 `DEV_NAME`，其 `flags` 参数设置为 `IORESOURCE_BUSY`。

因此一个 PCIe 设备的驱动程序使用 `pci_request_regions` 函数对 `pci_dev` 结构进行设置，将其使用的 `flags` 位设置为 `IORESOURCE_BUSY` 之后，其他驱动程序不能再次设置这个 `flags` 位。在实际应用中可能存在一个硬件设备对应两个设备驱动程序的情况，此时只有一个设备驱动程序可以使用 `pci_request_regions` 函数对资源进行管理。

`pci_resource_start` 函数从 `resource` 结构获得 BAR0 空间的基地址，该地址为存储器域的物理地址，而不是 PCI 总线域的物理地址，该值与直接使用 `pci_read_config_word` 函数读取 PCI 设备 BAR 寄存器所获得的值即便相等，也没有本质的联系，因为从 `resource` 结构获得的是该设备 BAR 寄存器在存储器域的物理地址，而使用 `pci_read_config_word` 函数获得的是 PCI 总线域的物理地址。在 Linux 驱动程序中，需要使用的是存储器域的物理地址。

本书从始至终一直强调 PCI 总线域物理地址和存储器域物理地址的区别，希望读者真正理解这两个地址的区别。在 x86 处理器中，并没有显式区分这两个物理地址的区别，这在某种程度上误导了部分系统程序员。

在这段程序的最后，使用 `ioremap` 函数将存储器域的物理地址映射成为 Linux 系统中的虚拟地址，之后 Capric 卡的设备驱动程序可以使用 `adapter->pci_bar0` 指针访问 Capric 卡中存储器映射的寄存器。

除了 `ioremap` 函数之外，Linux 系统还提供了 `ioremap_nocache` 和 `ioremap_cache` 函数用于存储器域虚实地址的转换。其中 `ioremap_nocache` 函数将存储器域的物理地址空间映射到一段“不可 Cache”的虚拟地址空间，在绝大多数体系结构中，这个函数与 `ioremap` 函数的实现一致，因为绝大多数外部设备都需要映射到“不可 Cache”的虚拟地址空间中。由于历史原因，99% 以上的程序员已经使用了 `ioremap` 函数而不是 `ioremap_nocache` 函数进行总线地址到虚拟地址的转换，因此 `ioremap_nocache` 函数显得冗余。

值得注意的是，对于 PCIe 设备的 Linux 驱动程序，`ioremap` 函数使用的物理地址必须从 `pci_resource_start` 函数获得，而不能使用“通过 `pci_read_config_xxxx` 函数”获得的 `BARx` 基地址，因为 PCIe 设备的 `BARx` 基地址空间属于 PCI 总线域，而不是存储器域。

某些设备还可能使用“可 Cache 的”虚拟地址空间，此时需要使用 `ioremap_cache` 函数

将存储器域地址转换为虚拟地址，目前为止，仅有极少数外部设备需要使用这一函数，如 PCIe 设备中的 ROM 空间。

Linux 系统还提供了一个 `ioremap_flags` 函数，使用这个函数可以自定义存储器域地址转换到哪种类型的虚拟地址，该函数提供了一个入口参数 `flags`，系统程序员可以使用该参数确定所申请虚拟地址空间的类型。对于 x86 处理器，`flags` 参数的定义见 `./arch/x86/include/asm/pgtable.h` 文件；而对于 PowerPC 处理器，`flags` 参数的定义见 `./arch/powerpc/include/asm/pgtable-ppc32.h` 文件。

源代码 12-5 Capric 卡的硬件初始化片段 4

```

    result = register_chrdev(test_dri_major, DEV_NAME, &capric_fops);
    ...

    result = pci_enable_msi(pci_dev);

    if(unlikely(result)) {
        PDEBUG("can not enable msi ... \n");
        goto chrdev_unregister;
    }

    result = request_irq(pci_dev->irq, capric_interrupt,
                        0, DEV_NAME, NULL);

    if(unlikely(result)) {
        PDEBUG("request interrupt failed ... \n");
        goto err_disable_msi;
    }
    ...

    capric_reset();
    return 0;
    ...
}

static const struct file_operations capric_char_fops = {
    .owner          = THIS_MODULE,
    .ioctl          = capric_ioctl,
    .open           = capric_open,
    .release        = capric_release,
    .write          = capric_write,
    .read           = capric_read,
};

```

这段源代码首先使用 `register_chrdev` 函数注册一个 `char` 类型的设备驱动程序，包括打开、关闭、读写操作和 `ioctl` 函数。之后该程序调用 `pci_enable_msi` 函数使能 Capric 卡的 MSI

中断请求机制，该函数将在第 12.3.5 节中详细介绍。

随后这段程序使用 request_irq 函数注册 Capric 卡使用的中断服务例程 capric_interrupt，并使用 pci_dev→irq 作为这个函数的 irq 入口参数。Capric 卡的 pci_dev→irq 参数在 Linux 系统对 PCI 总线进行初始化时分配，在 x86 处理器中，如果一个 PCIe 设备支持 MSI 中断，驱动程序执行完毕 pci_enable_msi 函数后，pci_dev→irq 参数还会发生变化。因此 request_irq 函数必须在 pci_enable_msi 函数之后运行。

这段源代码的最后将调用 capric_reset 函数，对 Capric 卡进行硬件初始化，该函数执行的操作见第 12.1.2 节。

12.3.3 Capric 卡的 DMA 读写操作

Capric 卡的 DMA 读/写过程与 capric_write/capric_read 函数对应。

1. DMA 写的操作流程

Capric 卡的数据传送方法较为简单，其 DMA 读写的硬件操作流程如第 12.1.3 和 12.1.4 节所示。DMA 写的实现过程与 capric_read 函数对应，如源代码 12-6~12-7 所示。

源代码 12-6 Capric 卡的 DMA 写片段 1

```
static ssize_t capric_read(struct file * file,
                          char _user * buff, size_t count, loff_t * f_pos)
{
    int err = -EINVAL;
    void * virt_addr = NULL;
    dma_addr_t dma_write_addr ;
    ...
    virt_addr = kmalloc(count, GFP_KERNEL);

    if((unlikely(! virt_addr))){
        PDEBUG("can not alloc rx memory you want ... \n");
        return -EIO;
    }

    dma_write_addr = pci_map_single(adapter->pci_dev,
                                    virt_addr, count, PCI_DMA_FROMDEVICE);

    if((unlikely(pci_dma_mapping_error(adapter->pci_dev, dma_write_addr))){
        PDEBUG("RX DMA MAPPING FAIL ... \n");
        goto err_kmalloc;
    }
}
```

这段源代码首先对 count 字段进行检查，因为 Capric 卡规定一次 DMA 操作所传递的数据不超过 2 KB，之后使用 kmalloc 函数分配 DMA 写使用的数据缓存。kmalloc 函数所能分配的内存大小受限于 Linux 系统中的 SLAB/SLUB 内存分配器，在系统内存紧张时，有可能失

败，因此在此必须进行参数检查。值得注意的是，在一个实际驱动程序中，很少在读写服务例程中使用 `kmalloc` 函数申请内存，然后使用 `kfree` 函数释放这段内存，因为这样做容易产生内存碎片。而且 `kmalloc` 函数的执行时间也相对较长，影响数据传送的效率。

随后这段代码调用 `pci_map_single` 函数将存储器域的虚拟地址 `virt_addr` 转换为 PCI 总线域的物理地址 `dma_write_addr`，供 Capric 卡的 DMA 控制器使用。Linux 系统提供了一组将虚拟地址转换为设备域物理地址的方法，参见第 12.3.5 节。

源代码 12-7 Capric 卡的 DMA 写片段 2

```

#ifdef CONFIG_NOT_COHERENT_CACHE⊖
    dma_sync_single( adapter->pci_dev,
                    virt_addr, count, PCI_DMA_FROMDEVICE);
#endif

    capric_w32( dma_write_addr, WR_DMA_ADR);
    capric_w32( count, WR_DMA_SIZE);
    capric_w32( MWR_START, DCSR2);

    if( unlikely( interruptible_sleep_on( adapter->dma_write_wait) ))
        goto err_pci_map;
    ...

    if( ( unlikely( copy_to_user( buff, virt_addr, count) )) )
        goto err_pci_map;

    pci_unmap_single( adapter->pci_dev, virt_addr, count, PCI_DMA_FROMDEVICE);
    kfree( virt_addr);
    return count;
    ...
}

```

如果当前 DMA 写操作不与 Cache 进行一致性操作，将首先执行 `dma_sync_single` 函数进行存储器与 Cache 的同步操作，该函数的详细说明见第 12.3.6 节。随后这段程序使用 `capric_w32` 函数执行第 12.1.3 节中要求的寄存器操作，之后可以使用轮询方式，或者使用中断方式唤醒这个 DMA 写进程。当进程被唤醒后，表示 DMA 写操作已经完成，此时这段程序使用 `copy_to_user` 函数将数据复制到用户空间。

值得注意的是，这段代码使用了 `interruptible_sleep_on` 函数将当前进程休眠，而在中断处理程序中使用 `wake_up_interruptible` 函数将其唤醒。这是一种非常糟糕的实现方式，而且存在相当大的隐患。

`interruptible_sleep_on` 函数的主要工作是将当前进程放入等待队列中睡眠，目前在 Linux 系统中，该函数已经逐步被 `wait_event_interruptible` 函数取代，但这并不是问题的关键。在源

⊖ `#ifdef` 和 `#endif` 出现在函数体中并不可取，请读者参阅 Greg Kroah-Hartman 的 Proper Linux Kernel Coding Style 掌握正确的处理方法。

代码 12-7 中，即便使用 `wait_event_interruptible` 函数也存在同样的问题。

因为 `interruptible_sleep_on` 函数的执行路径较长，很可能在当前进程还没有被该函数放入 `adapter->dma_write_wait` 队列时，处理器已经执行中断服务例程，打断 `interruptible_sleep_on` 函数，并执行 `wake_up_interruptible` 函数。Capric 中断服务例程的详细说明见第 12.3.4 节。

`wake_up_interruptible` 函数将唤醒在 `adapter->dma_write_wait` 队列中休眠的进程，而此时当前进程可能还没有被加入到等待队列中。当该函数执行完毕退出中断处理例程之后，处理器继续执行 `capric_read` 函数，并完成 `interruptible_sleep_on` 函数的执行，将自身加入到等待队列中睡眠。此时由于中断服务例程已经被提前执行，因此当前进程不会被 `wake_up_interruptible` 函数唤醒，从而造成死锁。

程序员可以使用 DCSR1 寄存器的 `msk` 和 `pending` 位解决这个死锁问题。采用这种方法时，设备驱动程序需要保证当前进程进入等待队列后，再允许 Capric 卡提交中断请求。但是这种方法将产生较长的中断延时，从而极大影响 Capric 卡的 DMA 读写效率。

程序员还可以使用 `interruptible_sleep_on_timeout` 或者 `wait_event_interruptible_timeout` 函数进行超时处理，使用该方法也可以解决上述死锁问题。

以上这两种方法都不是完美的解决方案，因为产生这种死锁的主要原因是 Capric 卡的逻辑设计并不合理。Capric 卡使用的数据传送模型较为简单，系统程序员很难基于此模型写出高效的驱动程序。

2. DMA 读的操作流程

Capric 卡 DMA 读使用的函数与 DMA 写的类似，其流程如源代码 12-8 所示。

源代码 12-8 Capric 卡的 DMA 读

```
static ssize_t capric_write(struct file *file,
                           const char __user * buff, size_t count, loff_t * f_pos)
{
    int err = -EINVAL;
    void * virt_addr = NULL;
    dma_addr_t dma_write_addr;

    virt_addr = kmalloc(count, GFP_KERNEL);
    ...
    if( ( unlikely( copy_from_user( virt_addr, buff, count) ) )
        return err;

    dma_write_addr = pci_map_single( adapter->pci_dev,
                                     virt_addr, count, PCI_DMA_TODEVICE );
    ...
#ifdef CONFIG_NOT_COHERENT_CACHE
    dma_sync_single( adapter->pci_dev, virt_addr,
                    count, PCI_DMA_TODEVICE );
```

```

#endif

capric_w32( dma_write_addr, RD_DMA_ADR);
capric_w32( count, RD_DMA_SIZE);
capric_w32( MRD_START, DCSR2);

adapter->dma_read_done = 0;

if( unlikely( interruptible_sleep_on( adapter->dma_read_wait) ) )
    goto err_pci_map;
...
pci_unmap_single( adapter->pci_dev,
                  dma_write_addr, count, PCI_DMA_TODEVICE);
kfree( virt_addr);
return count;
...
}

```

读者如果正确理解了上文关于 DMA 写的执行过程，DMA 读的执行过程并不难理解。Capric 卡 DMA 读的硬件操作流程如第 12.1.4 节所示。上述源代码使用 capric_w32 函数完成硬件寄存器的填写，然后可以使用轮询或者中断方式确定 DMA 读是否已经完成。在 DMA 读完成之后该例程将释放使用的内存资源后返回。与 DMA 写的操作流程类似，这段程序依然存在隐患。

12.3.4 Capric 卡的中断处理

Capric 卡一共需要处理三种中断请求，分别为 DMA 写完成、DMA 读完成和错误中断请求。Capric 卡使用了一个中断服务例程处理这些中断请求，其执行流程如源代码 12-9 所示。

源代码 12-9 Capric 卡的中断服务例程

```

static irqreturn_t capric_interrupt( int irq, void * dev)
{
    unsigned int statue;

    statue = capric_r32( INT_REG);

    if( ! ( statue & INT_ASSERT) ) {
        PDEBUG( "irq_none ... \n" );
        return IRQ_NONE;
    }

    if( statue & INT_ASSERT_R ) {

```

```

        capric_w32( statue, INT_REG );
        wake_up_interruptible( &adapter->dma_read_wait );
    }
    else {
        capric_w32( statue, INT_REG );
        wake_up_interruptible( &adapter->dma_write_wait );
    }

    PDEBUG( " irq handled ... \n" );
    return IRQ_HANDLED;
}

```

在 `capric_probe` 函数中, `capric_interrupt` 中断服务例程被 `request_irq` 函数注册到 Linux 系统的 `irq_desc` 中断描述符表中, 并与 Linux 系统的外部中断处理函数 `do_IRQ` 挂接, 当 Capric 卡通过 MSI 中断方式提交外部中断请求后, `do_IRQ` 函数将最终调用 `capric_interrupt` 函数完成相应的中断处理。Capric 卡处理中断请求的硬件操作流程如第 12.1.5 节所示。

目前 Linux 系统对 MSI 机制的支持并不理想, `pci_enable_msi` 函数^①仅可以获得一个 irq 号, 这为中断服务例程的设计带来了一定的困难。如果 `pci_enable_msi` 函数可以获得多个 irq 号, 那么在 `capric_probe` 函数中, 可以使用多个中断服务程序, 其中 DMA 写完成、读完成和错误处理分别使用三个中断服务例程, 而不必使用 `capric_r32` 函数读取 `INT_REG` 寄存器。在 Linux 系统中, 将 `pci_enable_msi` 函数改写为支持多个 irq 号并不困难。对于许多 PCIe 设备, 这种改写是必须的, 因为 RC 从 PCIe 设备中读取寄存器的代价是非常昂贵的。

12.3.5 存储器地址到 PCI 总线地址的转换

在 Linux 系统中, 支持一系列 API 实现存储器地址到 PCI 总线地址的转换, 这些 API 的详细定义见 `./Documentation/DMA-API.txt` 文件。本节仅以 `pci_map_single` 函数为例说明这种地址转换的工作原理。`pci_map_single` 函数在 `./include/asm-generic/pci-dma-compat.h` 文件中, 如源代码 12-10 所示。

源代码 12-10 `pci_map_single` 函数

```

static inline dma_addr_t
pci_map_single( struct pci_dev * hwdev, void * ptr, size_t size, int direction )
{
    return dma_map_single( hwdev == NULL ? NULL : &hwdev->dev,
        ptr, size, ( enum dma_data_direction ) direction );
}

```

该函数共有 4 个输入参数, 其中 `hwdev` 参数与 PCI 设备的 `pci_dev` 对应, `ptr` 参数对应存储器域的虚拟地址, `size` 字段对应数据区域的大小。而 `direction` 参数与数据区域的使用方法

① Linux 2.6.31 内核提供的 `pci_enable_msi_block` 函数也仅支持一个中断向量。

对应, PCI_DMA_NONE 用于调试, 较少使用; PCI_DMA_TODEVICE 表示这段数据的传递方向是从存储器到 PCI 设备; PCI_DMA_FROMDEVICE 表示这段数据的传递方向是从 PCI 设备到存储器; PCI_DMA_BIDIRECTIONAL 表示方向未知。该函数的返回值为 dma_addr, 即 PCI 总线域的物理地址。

pci_map_single 函数的主要作用是通过 ptr 参数, 获得与之对应的 dma_addr, 即进行存储器域虚拟地址到 PCI 总线域物理地址的转换。值得注意的是存储器域物理地址与 PCI 总线域物理地址的区别。

在 Linux 系统中, 使用 virt_to_phys 函数将存储器域的虚拟地址转换为存储器域的物理地址, 但是通过该函数仅能获得存储器域的物理地址, 因此该地址不能填写到 PCI 设备中进行 DMA 操作。值得注意的是, 进行 DMA 操作的地址是由 PCI 设备使用的, 而且这个地址只能是 PCI 总线域的物理地址, 尽管在许多处理器中, virt_to_phys 函数和 pci_map_single 函数的返回值相同。

不同的处理器使用不同的方式实现 pci_map_single 函数。起初在 x86 处理器中, 存储器域物理地址到 PCI 总线域物理地址的转换非常简单, 是直接相等的关系。但是 x86 处理器为了支持虚拟化技术, 使用了 VT-d/IOMMU^①技术, 使得该函数的实现略微复杂。

同样是基于 x86 架构, AMD 处理器使用的 IOMMU 技术与 Intel 有所区别, AMD 的 x86 处理器使用 ./arch/x86/kernel/amd_iommu.c 文件中的 map_single 函数, 进行存储器域地址空间到 PCI 总线域地址空间的转换; 而 Intel 的 x86 处理器使用 ./drivers/pci/intel-iommu.c 文件中的 intel_map_single 函数实现存储器地址空间到 PCI 域地址空间的转换。IOMMU 技术略微有些复杂, 在第 13.1 节中将专门描述这部分内容。

在 PowerPC 处理器中, 存在一组 Inbound 寄存器, 通过该组寄存器可以将 PCI 总线地址转换为 PowerPC 处理器规定的存储器地址, 详见第 2.2 节。这组 Inbound 寄存器也可以看作一种 IOMMU, 只是该 IOMMU 机制仅支持段式映射而不支持页式映射。

Linux PowerPC 使用 dma_direct_map_page 函数实现这个地址转换, 该函数的定义详见 ./arch/powerpc/kernel/dma.c。在 Linux PowerPC 中, PCI 总线域的物理地址也与存储器域的物理地址相等。

Linux PowerPC 还需要设置 Inbound 寄存器组, 这段代码在 ./arch/power/sysdev/fsl_pci.c 文件的 setup_pci_atmu 函数中, 如源代码 12-11 所示。目前这段代码对 Inbound 寄存器组的 Entry 2 进行设置, 允许 PCIe 设备访问 0~0x7FFF-FFFF (2 GB) 这段存储器域物理地址空间, 而且 PCI 总线地址与存储器地址一一对应而且相等。

源代码 12-11 setup_pci_atmu 函数

```
static void __init setup_pci_atmu(struct pci_controller *hose,
                                struct resource *rsrc)
{
    ...

    /* Setup 2G inbound Memory Window @ 1 */
```

① VT-d 是指 Intel 的 Virtualization Technology for Directed I/O 技术, 而 AMD 将这一技术称为 IOMMU。下文将这些技术都简称为 IOMMU。

```

out_be32(&pci->piw[2].pitar, 0x00000000);
out_be32(&pci->piw[2].piwbar, 0x00000000);
out_be32(&pci->piw[2].piwar, PIWAR_2G);
...
}

```

这段代码源于 Linux 2.6.30，在这个版本中，PCIe 设备不能访问 PowerPC 处理器 2 GB 之上的物理内存。而在 Linux 2.6.31.6 中，该函数被大规模修改，以支持超过 2 GB 的存储器系统，本节对 Linux 内核的这些改动不做进一步描述，对此有兴趣的读者可以参考 Linux 2.6.31.6 内核中 `setup_pci_atmu` 函数的最新实现。

有些支持 IOMMU 机制的 PowerPC 处理器，如 IBM 的 PowerPC 处理器系列，可以使用 `dma_iommu_map_page` 或者 `ibmebus_map_page` 函数实现 `pci_map_single` 函数，而 cell 处理器使用 `dma_fixed_map_page` 函数实现该功能。`pci_map_single` 函数在 IBM 的 PowerPC 处理器上已经移植完毕，但是 Freescale 除了 P4080 处理器之外，还没有支持 IOMMU 的处理器。目前对 P4080 处理器的支持并没有加入到 Linux PowerPC 中。

12.3.6 存储器与 Cache 的同步

Linux 系统还提供了一组 sync 函数，如 `dma_sync_single`、`dma_sync_sg` 等函数，这组 sync 函数的主要作用是为了支持“不进行 Cache 共享一致性”的 DMA 操作。

如果设备进行 DMA 操作时，不需要硬件进行 Cache 一致性操作^①，那么处理器在 DMA 操作之前，需要使用软件指令将操作的数据区域与 Cache 进行同步，之后进行 DMA 操作。PCIe 设备启动 DMA 请求时，如果其 TLP 头部 Attr 字段的 No Snoop Attribute 位为 1^②时，驱动程序也需要进行这种同步操作。在 PowerPC 处理器中，有一些非 PCIe 设备，如 QE (QUICC Engine) 中的一些内嵌设备，这些设备可以通过设置 snoop 位决定在 DMA 传送过程中，是否需要硬件进行 Cache 一致性操作。

目前多数 RC 或者 HOST 主桥都可以通过总线监听，解决 PCI 设备进行 DMA 操作的 Cache 一致性问题。但是有些 RC，如 MPC8572 处理器的可以通过设置 Inbound 寄存器决定当前访问是否支持 Cache 一致性操作。

如果硬件不支持 Cache 共享一致性，那么 PCI 设备进行 DMA 操作时，必须使用软件指令维护存储器与 Cache 的同步，从而避免 Cache 与主存储器不一致的现象发生。

系统软件程序员使用软件指令维护 Cache 时，务必深入理解这些指令的特点和使用方法。在处理器系统的设计中，有两类错误最难被发现，一类是 Cache 与存储器系统的不一致，另一类是数据传送的序引发的错误。

即使是对资深的系统程序员，也很难从这些错误表现形式中，发现是 Cache 不一致或者数据传送的序引发的系统错误。因此系统程序员需要重视在一个处理器系统中的 Cache 一致性 (Cache Coherency) 和数据完成性 (Data Consistency)。

① 有些处理器不支持硬件的 Cache 共享一致性，如一些低端的 ARM 处理器。

② 目前大多数 PCIe 设备进行 DMA 操作时，No Snoop Attribute 位都为 0。

因此虽然对于多数 PCI 设备，Cache 一致性可以由硬件保证，本节也必须讲述如何通过软件指令维护 Cache 的一致性。Linux 系统使用 `dma_sync_single` 函数维护 Cache 的一致性。`dma_sync_single` 函数的实现如源代码 12-12 所示。

源代码 12-12 `dma_sync_single` 函数

```
#define dma_sync_single dma_sync_single_for_cpu

static inline void
dma_sync_single_for_cpu(struct device * hwdev, dma_addr_t dma_handle,
                        size_t size, enum dma_data_direction dir)
{
    struct dma_map_ops * ops = get_dma_ops(hwdev);

    BUG_ON(! valid_dma_direction(dir));

    if (ops->sync_single_for_cpu)
        ops->sync_single_for_cpu(hwdev, dma_handle, size, dir);
    debug_dma_sync_single_for_cpu(hwdev, dma_handle, size, dir);
    flush_write_buffers();
}
```

不同的处理器系统使用不同的 `ops->sync_single_for_cpu` 操作函数。值得注意的是，Linux x86 并没有实现 `ops->sync_single_for_cpu` 函数，因为使用软件指令维护 Cache 一致性的情况在 x86 处理器系统中并不多见。而 Linux PowerPC 使用 `dma_direct_sync_single_range` 函数实现 `ops->sync_single_for_cpu` 函数，该函数最终将调用 `__dma_sync` 函数。这两个函数的实现如源代码 12-13 所示。

源代码 12-13 `__dma_sync` 函数

```
static inline void dma_direct_sync_single_range(struct device * dev,
                                                dma_addr_t dma_handle, unsigned long offset, size_t size,
                                                enum dma_data_direction direction)
{
    __dma_sync(bus_to_virt(dma_handle+offset), size, direction);
}
...

/*
 * make an area consistent.
 */
void __dma_sync(void * vaddr, size_t size, int direction)
{
    unsigned long start = (unsigned long)vaddr;
    unsigned long end = start + size;
```

```

switch ( direction ) {
case DMA_NONE:
    BUG();
case DMA_FROM_DEVICE:
    /*
     * invalidate only when cache-line aligned otherwise there is
     * the potential for discarding uncommitted data from the cache
     */
    if ( ( start & ( L1_CACHE_BYTES - 1 ) ) || ( size & ( L1_CACHE_BYTES - 1 ) ) )
        flush_dcache_range( start, end );
    else
        invalidate_dcache_range( start, end );
    break;
case DMA_TO_DEVICE: /* writeback only */
    clean_dcache_range( start, end );
    break;
case DMA_BIDIRECTIONAL: /* writeback and invalidate */
    flush_dcache_range( start, end );
    break;
}
}
EXPORT_SYMBOL(__dma_sync);

```

在 Linux PowerPC[⊖]中，flush_dcache_range、invalidate_dcache_range 和 clean_dcache_range 函数分别使用 dcbf、dcbi 和 dcbst 指令实现。dcbi/dcbf/dcbst 指令的格式为 dcbi/dcbf/dcbst rA, rB，如源代码 12-14 所示。

源代码 12-14 dcbi/dcbf/dcbst 指令格式

```

if rA = 0 then a <- 640 else a <- rA
EA <- 320 || ( a + rB ) 32:63
InvalidateDataCacheBlock( EA)           // dcbi
FlushDataCacheBlock                     // dcbf
StoreDataCacheBlock( EA)                 // dcbst

```

dcbi、dcbf 和 dcbst 指令的详细说明如下。

- dcbi 指令首先在 Cache 中检查 EA。如果 EA 在 Cache 中命中，将直接将 EA 所对应的 Cache 行的状态改变为 I，无论这个 Cache 行原来的状态是什么，都不将数据回写到存储器中。
- dcbf 指令首先在 Cache 中检查 EA。如果 EA 在 Cache 中命中，则继续检查 Cache 的状态，如果为 M，则将 Cache 行刷新到内存，然后 Invalidate 该 Cache 行，将 Cache 行的

⊖ 以 E500 V2 内核为例。

状态改变为 I；否则直接 Invalidate 该 Cache 行。

- dcbst 指令首先在 Cache 中检查 EA。如果地址在 Cache 中命中，则继续检查 Cache 的状态，如果为 M，则将 Cache 行回写到内存，然后将 Cache 行的状态改变为 E；否则不做任何操作。

在 x86 处理器系统中，也存在类似的 Cache 指令。如 INVD、WBINVD 和 CLFLUSH 指令。其中 INVD 指令的作用是 Invalidate 处理器中的内部 Cache，并通过 FSB 总线周期 Invalidate 外部 Cache；而 WBINVD 指令是在 Invalidate 内部和外部 Cache 之前，先将 Cache 中的数据回写，然后进行 Invalidate 操作。

但是这两条指令都是针对整个 Cache，而不是针对某个 Cache 行。如果需要对 Cache 行进行刷新时，x86 处理器可以使用 CLFLUSH 指令操作某个 Cache 行，该指令所实现的功能与 dcbf 指令类似。单从操作 Cache 行的指令的角度上看，PowerPC 处理器比 x86 处理器好得多，因此本节以 PowerPC 处理器为例说明这些 Cache 指令在不同情况下的使用方法。

下文将分别介绍在 DMA 写和 DMA 读操作中 __dma_sync 函数的工作流程。假设在一个单处理器系统[⊖]中，Cache 行长度为 512b，而且 PCI 设备进行 DMA 读写操作时，硬件不进行 Cache 一致性操作。

1. DMA 写

在外部设备进行 DMA 写操作之前，需要使用 __dma_sync 函数同步 Cache 与存储器中的数据。有许多书籍包括 Linux 系统中的 DMA-API 文档，都认为在 DMA 写操作完成后，调用 __dma_sync 函数 Invalidate 数据区域所对应的 Cache 行，处理器就可以使用来自设备的数据。这种说法是基于设备访问的数据区域头尾都是 Cache 行对界的情况而言的，如果数据区域并不是 Cache 行对界时，这种做法将引发系统错误。

假设在一个处理器系统中，Cache 行长度为 64B。当一个 PCI 设备通过 DMA 写操作，访问 0x1001~0x10FE 这段数据区域时，这段数据区域将占用 4 个 Cache 行，而且并不是 Cache 行对界的，如图 12-9 所示。

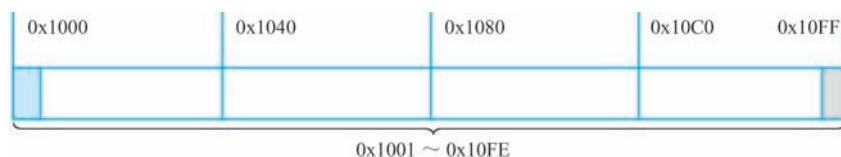


图 12-9 DMA 写访问的数据区域不对界

如果在 0x1000~0x10FF 这段数据区域中，0x1000 和 0x10FF 字节曾经被改写过，那么 0x1000~0x103F 和 0x10C0~0x10FF 这两个 Cache 行的状态为 M，因此图 12-9 中阴影部分的数据与存储器不一致，而且为处理器系统中最新的数据。

而 DMA 写结束后，0x1001~10FE 这段数据区域被 PCI 设备改写，且为处理器系统中最新的数据，此时这段数据区域对应的 Cache 行状态仍然为 M。此时如果处理器 Invalidate 0x1001~0x10FE 这段数据区域对应的 Cache 行，即 Invalidate 0x1000~0x10FF 这段数据区域的 Cache 行时，将会丢失 0x1000 和 0x10FF 这两个字节中保存的合法数据。如果处理器刷新

⊖ 在 SMP 系统中，Cache 行的状态转换更为复杂。

0x1001~0x10FE 这段数据区域对应的 Cache 行，即刷新 0x1000~0x10FF 这段数据区域的 Cache 行时，将丢失所有来自 PCI 设备的数据，采用这种方法问题更大。

通过上述分析可以发现，如果在 DMA 写完成后，再对访问的数据区域进行 Cache 同步操作，将可能引发严重的 Cache 一致性问题，从而导致整个系统异常。

为此正确的方法是在 DMA 写操作之前，将其访问的数据区域与 Cache 进行一致性操作，如源代码 12-13 中“case DMA_FROM_DEVICE”所示。但是这段源代码仍然有较大的问题，因为这段代码在处理数据区域不对界的情况时，将刷新整个数据区域对应的 Cache 行。

这种做法不会产生错误，但是会影响效率。假设与 0x1000~0x10FF 这段数据区域对应的 Cache 行的状态都为 M，那么这段程序将 0x1000~0x103F、0x1040~0x107F、0x1080~0x10BF 和 0x10C0~0x10FF 对应的 Cache 行都刷新到存储器中，然后再 Invalidate 这些 Cache 行。而实际上 0x1040~0x10C0 这段数据区域将由 PCI 设备重新填写，因而将这部分区域进行刷新然后再 Invalidate 是没有意义的。

正确的做法是刷新不对界的数据区域 0x1000~0x1040 和 0x10C0~0x10FF，即将这段数据区域的头尾刷新即可，而直接 Invalidate 中间的数据区域 0x1040~0x10BF。采用这种方法将有效地提高系统效率。

2. DMA 读

外部设备进行 DMA 读操作之前，处理器必须保证该设备访问的数据区域与 Cache 一致。因此需要调用 __dma_sync 函数（dir 参数为 DMA_TO_DEVICE）将 Cache 中的数据回写到存储器。该函数执行完毕后，Cache 中的所有数据都与存储器一致，而之前状态位为 M 的 Cache 行将更改为 E。经过这个 Cache 同步操作后，设备进行 DMA 读操作就可以从存储器中获得正确的数据。

此处还有一个细节问题值得考虑，就是 DMA 读操作是调用 dcbst 指令回写 Cache 行，还是调用 dcbf 指令刷新 Cache 行。如上文所述 dcbf 和 dcbst 指令都将状态位为 M 的 Cache 行与存储器进行同步，只是 dcbf 将 Cache 行的 M 位更新为 I，而 dcbst 指令将 Cache 行的状态更新为 E。使用这两个指令都可以保证 DMA 读的数据不会出现一致性问题。

此时在处理器系统中，如果 DMA 读的数据将不会被处理器使用时，应该使用 dcbf 指令，Invalidate 该 Cache 行，从而该 Cache 行可以被其他进程使用；如果 DMA 读的数据将很快被处理器使用时，应该使用 dcbst 指令，回写该 Cache 行，从而处理器使用该数据时，可以从 Cache 而不是存储器中获得。

12.4 Capric 卡的延时与带宽

总线的延时与带宽是一个巨大的话题，即便本节将其局限到 PCIe 总线，局限到 Capric 卡，也并不能改变这个话题内容的繁多。总线的带宽和延时之间有一定的制约关系，任何一个处理器系统都希望在获得巨大总线带宽的同时，尽量缩小访问延时。

在处理器系统的设计中，必须权衡“提高带宽”与“缩小延时”之间的关系，以组建一个合理的应用系统。在处理器领域，带宽是指数据的传送速率，即 1 秒钟传送的数据大小。而延时指传送处理一个数据单元所需要的时间。

片面地追求带宽显然并不合理，过大的带宽对某些应用并不合适。我们无法忽视一个满

载光盘的火车所能提供的带宽，这辆火车至少能传递几十万 Pebibyte 大小的数据，而且十个小时左右就能从北京抵达上海。显然多辆火车所能提供的带宽非常巨大，但是我们仍然无法使用火车传送网络报文，因为十个小时的延时是许多系统应用无法容忍的。

延时与带宽之间存在某种联系，而不是直接对立关系，并不是带宽越大延时也越大，带宽越小延时也越小。处理器系统设计所追求的目标是提高带宽的前提下，尽可能掩盖传送延时，组成一个可实现的处理器系统。

PCIe 总线作为处理器的通用局部总线，需要权衡带宽和延时之间的关系，满足在处理器系统中多数应用的需求，而非包罗万象。PCIe 总线能够满足，绝大多数处理器系统，特别是 PC 系统中的显卡、硬盘，声卡和一些慢速设备与处理器之间的数据传送，但是对于一些延时要求较高的应用并不适用。

当一个网络设备需要以 1 Gb/10 Gb 的速率“线速传送”64B 大小的网络数据报文时，PCIe 总线显得力不从心，通常这种网络设备需要直接与处理器的 FSB 相连，以尽量缩短传送延时。在许多处理器系统中，如 Freescale 的 P4080 处理器，RMI 的 XLP832、Cavium 的 CN6335 处理器，网络设备与 FSB 直接相连，并在追求最大带宽的同时，尽量减少访问延时。

在 x86 处理器系统中，可以使用 QPI（Quick Path Interconnect）连接这样的高性能网卡。但是目前在 PC 领域中，高性能网卡依然使用 PCIe 总线进行连接。

在一个处理器系统中，为掩盖传送延时，通常使用“流水线”技术，当数据传送的延时增加时，“流水线”所使用的资源也随之增加，并很容易到达处理器系统所不能忍受的范围。本章讲述在 PCIe 总线中，延时与带宽间的关系，以及存在的问题。PCIe 总线基于 TLP 进行数据传递，因此本节所强调的带宽与延时与 TLP 直接相关。

在本节中，PCIe 总线的带宽指每秒钟传送的“TLP 中有效数据”的大小，即 PCIe 总线的有效带宽。该定义与 PCIe 总线的链路带宽不同。如 PCIe V2.1 总线规范链路带宽为 5 GT/s，而这个带宽需要去掉物理层 8/10b 转换，以及 PCIe 总线的协议开销后才能得到 PCIe 总线的有效带宽。PCIe 总线的有效带宽与许多因素相关，包括协议开销、TLP Payload 的大小、传送延时、流量控制等因素相关，当然最重要的因素依然是 PCIe 总线的链路宽度。

PCIe 总线的延时指一个存储器请求从产生到结束的时间。值得注意的是存储器读与存储器写 TLP 的延时计算有所不同，存储器写 TLP 产生于发送端而结束于接收端，仅计算单向延时；而存储器读请求 TLP 产生于发送端，接收端将存储器读请求 TLP 转换为存储器读完成 TLP，再发送给发送端，需要计算双向延时。

12.4.1 TLP 的传送开销

在 PCIe 总线中，TLP 产生于事务层，并在通过 PCIe 总线的链路层与物理层时，加入若干前缀和后缀后，才能经由 PCIe 端口发送。PCIe 总线定义了多种 TLP，本章重点关心与存储器读写相关的 TLP，包括存储器写 TLP、存储器读请求 TLP 和存储器读完成 TLP。这些 TLP 报文的通用格式如图 12-10 所示。

1 Byte	2 Byte	3 ~ 4 DW	0 ~ 1024 DW	1 DW	1 DW	1 Byte
Start	Sequence ID	TLP Head	Data Payload	ECRC	LCRC	End

图 12-10 PCIe 总线 TLP 格式

由上图所示，一个 PCIe 设备发送 TLP 报文时，在经过数据链路层和物理层时，需要加上若干前缀和后缀。

1) Start 和 End 前后缀由物理层添加，表示一个 TLP 的开始与结束，各由一个字节组成，用于物理层同步 TLP 发送与接收。

2) Sequence ID 和 LCRC 前后缀由数据链路层添加，存放 TLP 的识别号和数据链路层的 CRC 校验，分别由 2B 和 1DW 组成。

3) TLP Head 前缀描述 TLP 的属性，由 3~4 个 DW 组成。

4) ECRC 后缀存放 TLP 在事务层的 CRC 校验，该字段可选。

5) 而 Data Payload 是真正有效负载，其长度在 0~1024DW 之间。由上文所述，TLP 的有效负载长度由 PCIe 设备的 Max_Payload_Size 参数确定，该字段通过上下游链路进行协商后获得。目前在多数处理器系统中，PCIe 设备使用的 Max_Payload_Size 参数为 128B 或者 256B。

我们假设 TLP 使用 3DW 的报文头，而且不需要 ECRC 校验，根据图 12-10，在一个 TLP 中，Data Payload 所占的比例如式 12-11 所示。

$$\text{Payload_Ratio} = \text{Payload} / (\text{Payload} + 20) \quad (12-11)$$

假设 PCIe 设备的 Max_Payload_Size 参数为 256B 时，根据以上公式可以得出该 PCIe 设备最大的 Payload_Ratio = $256 / (256 + 20) \approx 92.8\%$ 。但是 TLP 在 PCIe 链路中进行传送时，远不能获得 Payload_Ratio 大小的链路带宽。TLP 在传送过程中，需要通过 PCIe 总线的事务层、数据链路层和物理层，因此必须考虑这些协议所带来的开销。

1. 事务层的开销

事务层的开销需要分存储器写和存储器读两种情况讨论。在 PCIe 总线中，存储器写请求 TLP 使用 Posted 总线事务，而存储器读请求 TLP 使用 Split 总线事务。在这两种情况之下，事务层的开销不同。

PCIe 设备进行 DMA 写的过程较为简单，当 PCIe 设备将一个 4 KB 大小的数据传送到存储器时，首先将 4 KB 数据封装到多个存储器写请求 TLP 中，然后发向 RC，其中每个 TLP 最大的 Payload 为 Max_Payload_Size。

PCIe 设备的 Max_Payload_Size 参数由 PCIe 链路协商确定，目前 x86 处理器系统 RC 的 Max_Payload_Size 为 128B 或者 256B，所以与 RC 直接相连的 PCIe 设备，其 Max_Payload_Size 参数只能为 128B 或者 256B。由公式 12-11，可以计算出与 Max_Payload_Size 参数对应的 Payload_Ratio，并由此推算存储器写 TLP 在事务层上的开销。

PCIe 设备进行 DMA 读的过程略微复杂。首先 PCIe 设备向 RC 发送存储器读请求 TLP，当 RC 收到这个存储器读请求 TLP 后，将从存储器中获得数据，然后组成一个或者多个存储器读完成 TLP，并将其传送给 PCIe 设备。

存储器读完成 TLP 能请求的数据大小为 MRRS (Max_Read_Request_Size)，该参数的大小为 128B~4096B。如果 PCIe 设备进行 DMA 读的大小超过该参数时，将以该参数为界，向 RC 发出多个存储器读请求 TLP。而 RC 可以使用一个存储器读完成报文传递所有数据；也可以以 RCB 为边界，使用多个存储器读完成报文传递所有数据。大多数 RC 使用后一种方式传递存储器读完成 TLP，而且一次存储器读完成报文的大小也不超过 RCB。本章以这种方式为例分析存储器读 TLP 在事务层中的开销。

假设存储器读请求 TLP 头的大小为 3DW，而且报文头中不包含 ECRC 校验。PCIe 设备进行 DMA 读的大小恰好为 MRRS 时，RC 需要使用 MRRS/RCB 个存储器读完成报文传递数据。此时一次 DMA 读操作中 Data Payload 所占的比例如式 12-12^①所示。

$$\text{Payload_Ratio} = \text{MRRS} / (\text{MRRS} + 3 \times 4 + 3 \times 4 \times \text{MRRS} / \text{RCB}) \quad (12-12)$$

当 MRRS 为 512B，而 RCB 为 64B 时， $\text{Payload_Ratio} = 512 / (512 + 12 + 12 \times 8) \approx 82.6\%$ 。由以上分析可以发现 PCIe 设备进行 DMA 读在事务层上的开销大于 DMA 写在事务层上的开销，这也是 PCIe 设备 DMA 写的速度略高于 DMA 读的主要原因。

除了事务层的开销之外，DMA 读操作的数据传送路径也长于 DMA 写，因而访问延时大于 DMA 写操作的访问延时，这也为 DMA 读逻辑的设计带来了不小的麻烦。从第 12.2.2 节中，也可以发现 DMA 读逻辑的设计远比 DMA 写逻辑的设计复杂。

2. 链路层的开销

由图 12-10 所示，链路层向 TLP 添加 Sequence ID 和 LCRC 前后缀，这些开销已经在上文中计算，本小节不再重复计算这些开销。本小节所关心的链路层开销由两部分组成，一个是 ACK/NAK 协议的开销，另一个是流量控制所带来的开销。如第 7.2 节所示，发送方在发送 TLP 时，首先将这些 TLP 放入到 Replay Buffer 中，直到收到接收方的 ACK 报文后，才能确认该 TLP 已经正确地接收方接收；如果收到接收方 NAK 报文，则表示部分 TLP 没有被正确接收，需要重新发送这些 TLP。这些 ACK/NAK 报文将占用部分链路带宽。

在 PCIe 设备的实现过程中，设计者可以调整 TLP 接收个数的阈值，这个阈值的定义为接收端收到多少 TLP 后，给发送端提供一次 ACK/NAK DLLP。该阈值决定了数据接收端发送 ACK/NAK 报文的间隔。

该阈值越大，则发送端的 Replay Buffer 也将随之增大，否则发送端无法将数据及时填入 Replay Buffer，从而阻塞了发送流水，并影响 PCIe 总线的传送效率；如果该阈值越小，则接收端需要发送较多的 ACK/NAK 报文给接收端，也会影响 PCIe 总线的效率。因此接收端需要合理地设置 ACK/NAK 的阈值，以最大限度地利用 PCIe 总线的带宽。

在链路层的设计中，需要选择合适的 Replay Buffer 的大小。如果 Replay Buffer 过小，事务层无法及时地将 TLP 发送到 Replay Buffer，从而造成 TLP 发送流水线的中断。而保存在 Replay Buffer 中的报文需要得到对端设备的确认报文后，才能释放。

因此可以发现 Replay Buffer 的大小，只需要保证事务层发送 TLP 时，有足够的缓冲即可。Replay Buffer 的大小与 PCIe 链路的延时相关。在实现中，Replay Buffer 不能过大，否则将使用较多的芯片资源。

在链路层中，除了 ACK/NAK 报文的开销外，流量控制报文也需要占用 PCIe 总线的链路开销。PCIe 总线使用 Credit-Based 流量控制策略，发送端需要保证接收端有足够的缓冲之后才能发送报文，而且接收端需要按照某种策略及时使用 FC Update 报文，向发送端通知剩余的数据缓冲。因此流量控制也需要占用一些 PCIe 总线的带宽。在 PCIe 总线中，流量控制是基于“端到端”的，而且 PCIe 总线并没有规定 PCIe 设备使用的流量算法，因此流量控制对 PCIe 总线带宽的影响与设备相关，并没有一个统一的公式。

① 该公式没有计算物理层和数据链路层报文头的开销。

3. 物理层的开销

在 PCIe V2.1 总线规范中, TLP 在物理层中还需要进行 8/10b 转换, 这个转换将极大地降低 PCIe 总线的实际链路带宽, 而且在 PCIe 总线中, 这种带宽的浪费是无法避免的。在 PCIe V3.0 规范中, 这个 8/10b 转换被升级为 128/130 转换。128/130 转换将极大节约 PCIe 链路带宽的浪费。但是无论如何, TLP 在发送过程中, 仍然会因为这种转换浪费 PCIe 链路的一些带宽。

除了 8/10b 转换之外, 物理层为了解决接收时钟与逻辑时钟间的漂移所带来的问题, 每一个 Lane 需要在发送 1180~1538 个字符后, 发送一个 SKIP 序列进行时钟补偿。这种定时的时钟补偿序列也将浪费 PCIe 链路的部分带宽。

12.4.2 PCIe 设备的 DMA 读写延时

上节简要介绍了影响 PCIe 设备进行数据传递的因素, 无论设计者采用什么样的设计方式, TLP 在通过事务层、链路层和物理层时都会受到这些因素的影响。但是不同的设计方法依然会极大影响 PCIe 总线的使用效率。

本文中出现的 Capric 卡是一个很糟糕的设计, 在这个设计中, 并没有使用流水线机制来掩盖 PCIe 总线的延时, 因此该卡通过 PCIe 总线进行 DMA 读时的效率并不高。

1. Capric 卡 DMA 写的效率

在 Capric 卡中, DMA 写操作由多个步骤组成, 并由 Capric 卡的硬件逻辑和处理器的中断处理机制协调完成, 其步骤如图 12-11 所示。

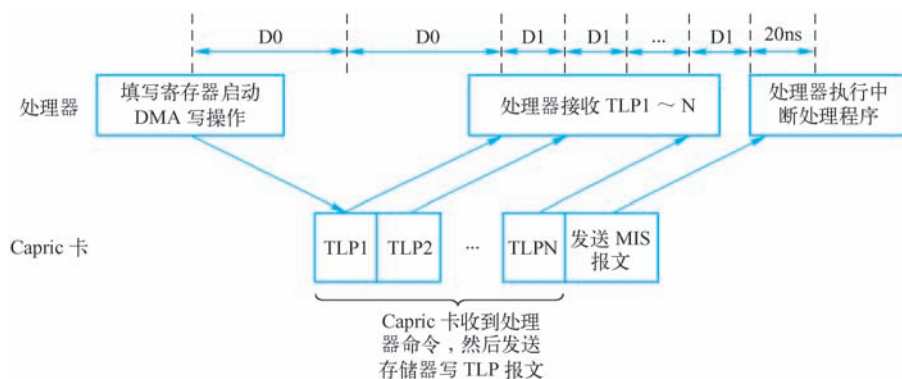


图 12-11 Capric 卡的 DMA 写过程

首先处理器填写 Capric 卡的 WR_DMA_ADR、WR_DMA_SIZE 和 DCSR2 寄存器, 经过延时 D0 之后, 这些命令陆续到达 Capric 卡。其中 D0 的大小与 Capric 卡连接在处理器系统中的位置相关, 而与 Capric 卡的设计无关。如果 Capric 卡直接与 RC 相连接, 则 D0 的值较小; 如果 Capric 卡通过多级 Switch 之后再与 RC 连接, 则 D0 较大。

Capric 卡收到处理器的 DMA 写请求后, 将向 RC 连续发送存储器写 TLP, 并由 RC 将数据写入到主存储器。Capric 卡根据 WR_DMA_SIZE 寄存器的数值, 将数据分解为多个存储器写请求 TLP, 其中每个存储器写请求 TLP 的有效负载不超过 Max_Payload_Size 参数, 本节假设该参数的大小为 128B。

假定 Capric 卡使用 100 MHz^①的时钟，而内部总线宽度为 64 位，此时 Capric 卡内部总线的带宽可以达到 800 MB/s，该值非常接近×4 PCIe 链路所能提供的有效带宽；存储器写 TLP 使用 3DW 的报文头且不使用 ECRC 校验，而每个存储器写 TLP 的最大有效负载为 128B (32DW)。因此在 Capric 卡中，一个存储器写 TLP 的最大长度为 35DW，此时 Capric 卡需要使用 18（实际值为 17.5）个时钟周期才能将这个 TLP 发送出去，因此在 Capric 卡中 D1 的大小为 180 ns。

Capric 卡将存储器写 TLP 发送完毕后，将向 RC 发送 MSI 报文，MSI 报文也是一种存储器写 TLP，Capric 卡使用两个时钟周期，即 20 ns 即可将该报文发送出去。RC 在等待一段延时后，将陆续收到存储器写请求 TLP1~N 和 MSI 报文，这段延时为 TLP 从 EP 到 RC 的延时，约等于 D0。

处理器收到 MSI 报文后，将执行中断处理程序，Capric 卡的中断处理例程通过 RC 读取中断控制状态寄存器 INT_REG，并结束整个 DMA 写操作。HOST 处理器读取中断控制状态寄存器的开销绝对不能忽略，因为这个读取过程首先是 RC 发送存储器读请求 TLP，当 Capric 卡收到这个 TLP 后再向 RC 发送存储器读完成 TLP，其访问延时为 2×D0。

假设 Capric 卡一次 DMA 写的大小为 X 字节^②，则这次 DMA 写所需的时间 T_{dmaw} 如式 12-13 所示。

$$T_{\text{dmaw}} = D0 + D0 + X/128 \times D1 + 20 + 2 \times D0 \quad (12-13)$$

其中 T_{dmaw} 的值越小，Capric 卡传送 X 字节的数据所需的时间也越短。但是以上公式并没有考虑 Capric 在 DMA 写过程中，因为数据缓冲不足而暂时中断存储器写 TLP 发送流水线的情况，而且忽略了中断处理例程所需的切换与执行时间。

由以上公式，可以发现当 D0 越大 T_{dmaw} 也越大。但是当 X 越大时，D0 在 T_{dmaw} 中所占的比重越小，在一个处理器系统中，D0 的大小是 Capric 卡无法控制的，该值的大小与 Capric 卡在处理器系统的位置和处理器系统的 RC 确定，属于系统延时，我们假设该值为 250 ns^③。根据以上假设，可以利用公式 12-13 获得 Capric 卡 DMA 写的有效带宽，如表 12-3 所示。

表 12-3 X 的大小与 DMA 写有效带宽的关系

T_{dmaw} (ns)	X	Capric 卡的最大有效带宽
1200	128B	106.7 MB/s
1380	256B	185.5 MB/s
1740	512B	294.3 MB/s
2460	1 KB	416.3 MB/s
3900	2 KB	525.1 MB/s
6780	4 KB	604.1 MB/s
12540	8 KB	653.2 MB/s
24060	16 KB	681.0 MB/s

① 实际上 LogiCORE 只能使用 100 MHz 或者 250 MHz 的内部时钟，但是笔者没有办法在 FPGA 内部运行 250 MHz 的时钟频率。

② X 可以被 128 整除。

③ 在 Tylersburg EP 平台中，这个系统延时大于 250 ns。

由表 12-3 所示，X 越大，Capric 卡的有效带宽也越高，因此适当提高 X 的大小将有效提高 Capric 卡 DMA 写的传送效率。

2. Capric 卡 DMA 读的效率

在 Capric 卡中，DMA 读的过程比 DMA 写过程略微复杂一些。因为 DMA 读是由两部分组成的，首先 Capric 卡向 RC 发起存储器读请求 TLP；当 RC 从存储器获得数据后，再向 Capric 卡发送存储器读完成 TLP，其实现过程如图 12-12 所示。

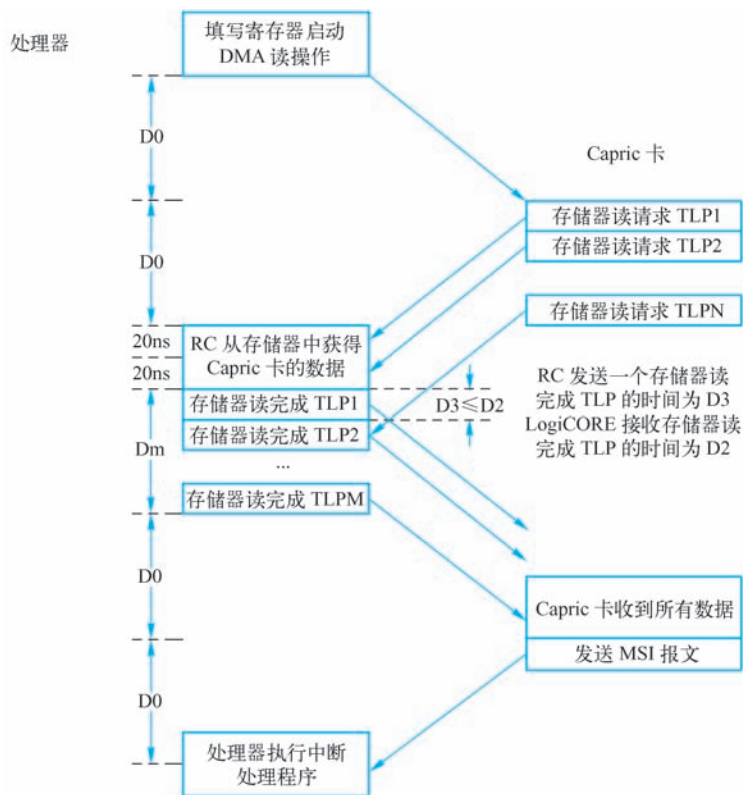


图 12-12 Capric 卡的 DMA 读过程

如上图所示，Capric 卡的 DMA 读过程如下所示。

1) 首先处理器填写 Capric 卡的寄存器启动 DMA 读。

2) Capric 卡在等待 D0 这段延时后，收到这个命令，之后向 RC 提交存储器读请求 TLP。假设 Capric 卡一共需要 N 个存储器读请求 TLP 才能发送完成整个请求，而且需要 D_n 这段时间才能将完成这些操作。而且存储器读请求 TLP 到达 RC 的延时也为 D0。

3) 处理器在等待 D0+D0 这段延时后，将开始接收存储器读请求 TLP，并从主存储器获得相应的数据，并组织存储器读完成报文发向 Capric 卡。RC 从开始接收存储器读请求 TLP 到接收最后一个存储器读请求 TLP 的时间延时为 D_n。其中 D_n 也与 Capric 卡发送全部存储器读请求的延时相同。

4) RC 收到来自 Capric 卡的存储器读请求 TLP 后，开始发送存储器读完成报文，其中 Capric 卡接收存储器读请求 TLP 与 Capric 卡发送存储器读完成 TLP 可以同步进行，因此 D_n 这段延时并不会被重复计算，而仅计算发送存储器读完成报文的流水准备时间，这段时间由

两部分组成, Capric 卡发送一个存储器读请求 TLP 的延时, 和 RC 从主存储器中读取数据的延时。我们假定这两段延时都为 20 ns, 因此流水准备时间为 40 ns。

5) Capric 卡经过 D0 这段延时后开始接收存储器读完成 TLP, 并经过 Dm 这段延时后将接收完毕所有来自 RC 的存储器读, 并获得 DMA 读的所有数据。

6) Capric 卡发送 MSI 报文通知处理器 DMA 读完成。

7) 处理器经过 D0 这段延时后, 收到 MSI 报文, 并开始执行中断处理程序。如果这个中断处理程序仍然需要读取中断控制状态寄存器, 则处理器需要 $2 \times D0$ 这段延时时间之后才能完成 Capric 卡的 DMA 读。

其中 D3 延时为 RC 发送存储器读完成 TLP 所需要的时间, D3 延时与 RC 的 RCB 参数相关。我们假设 RC 的 RCB 参数为 64B, 存储器读完成中的 Payload 为 64B, 而读完成报文头为 12B; 同时假设 RC 使用 128B 的数据总线, 而且其总线频率为 667 MHz, 此时 RC 发送的数据报文已经超过了 Capric 卡接收存储器读完成报文的速度, 因此在分析中我们使用 D2 延时, 其中 D2 为 Capric 接收存储器读完成报文的延时。Capric 卡接收一个长度为 84B 的报文时间约为 110 ns (实际需要 10.5 个时钟周期)。因此 Dm 的计算方法如式 12-14 所示。

$$Dm = D2 \times X / 64 = 110 \times X / 64 \quad (12-14)$$

由以上分析, 可以发现在 PCIe 总线中, Capric 卡 DMA 读比 DMA 写的过程略微复杂。假设 Capric 卡一次 DMA 读的大小为 $X^\ominus$ 字节, 则这次 DMA 读所需的时间 T_{dmar} 如式 12-15 所示。

$$T_{dmar} = 2 \times D0 + 40 + 110 \times X / 64 + 2 \times D0 + 2 \times D0 \quad (12-15)$$

由以上公式, 可以发现当 D0 越大时 T_{dmar} 也越大, 但是当 X 越大时, D0 在 T_{dmar} 中所占的比重越小。假设 D0 为 250 ns 时, 可以根据公式 12-13 获得 Capric 卡 DMA 读的有效带宽, 如表 12-4 所示。

表 12-4 X 的大小与 DMA 读有效带宽的关系

T_{dmar}	X	Capric 卡的最大有效带宽
1760 ns	128B	72.7 MB/s
1980 ns	256B	129.3 MB/s
2420 ns	512B	211.6 MB/s
3300 ns	1 KB	310.3 MB/s
5060 ns	2 KB	404.7 MB/s
8580 ns	4 KB	477.4 MB/s
15620 ns	8 KB	524.4 MB/s
29700 ns	16 KB	551.6 MB/s

由表 12-3 和表 12-4 可以发现, Capric 卡的 DMA 写的效率略高于 DMA 读的效率。以上有关 Capric 卡 DMA 读写效率是一个粗粒度的分析, 其分析结果并没有考虑数据链路层、物理层和流量控制的开销, 也没有考虑发送接收流水线中断的情况, 是一个较为理想的结

$\ominus$ X 可以被 64 整除。

果。笔者的实测结果与表 12-3 和表 12-4 有些差距，因为在 x86 处理器系统中，D0 延时时间远大于 250 ns。

12.4.3 Capric 卡的优化

由上两节的分析可以发现，制约 Capric 卡 DMA 读写带宽的主要因素由两部分组成，一是 PCIe 总线的链路带宽，二是在数据传送过程中的延时。使用更宽的 PCIe 链路，显然可以增加带宽。Capric 卡可以使用×8 的 PCIe 链路进一步提高物理带宽。但这并不是本章所侧重的提高物理带宽的方法，因为在一个给定的处理器系统中，PCIe 链路的带宽是一定的，设计者已经充分考虑了这些带宽。本章所侧重的是通过减少系统延时以提高带宽。

1. 减少处理器对 Capric 卡的寄存器读操作

由上文的分析可以发现，处理器读取 Capric 卡的寄存器需要通过两个步骤，首先发送存储器读请求 TLP，EP 接收到这个读请求后，再向 RC 发送存储器读完成 TLP。这个读操作的操作延时为 2×D0，这个延时是处理器系统所无法承受的，更为重要的是这个延时将严重阻塞设备进行 DMA 读写操作的流水线。

为此 Capric 卡将处理器需要读取的数据，以 DMA 写的方式预先写入到主存储器，之后处理器只需要读取主存储器即可获得相应的信息。处理器读取主存储器的延时与读取 EP 中的寄存器相比，可以忽略不计。而 Capric 卡进行 DMA 写这段延时，可以掩盖在数据传送的流水中，从而不会影响数据传送的效率。

在上文中，处理器在执行中断处理程序时，还需要读取中断控制状态寄存器。这个读取寄存器的开销也是可以避免的，因为 MSI/MSIX 中断机制支持“Multiple Message”。在 Capric 卡的实现中，发送完成、接收完成和错误处理中断请求可以与独立的中断服务例程对应。因此可以有效避免在发送完成、接收完成中断服务例程中读取中断控制状态寄存器，从而减少系统的延时。

2. 流水线技术

使用流水线技术可以有效地掩盖数据传送中的延时，从而提高带宽。在 PCIe 设备中，通常使用 Ring Buffer 技术，实现多路 DMA 读写操作的并发执行，从而显著提高 DMA 读写的效率。在 Cornus 卡中使用了 Ring Buffer 技术，与 Capric 卡相比，极大提高了 DMA 读写效率。采用 Ring Buffer 的这种方式也称为 Ring-Based DMA 机制。

目前 Ring Buffer 技术在网络设备中得到了广泛的应用，在此类网卡中，分别为发送部件和接收部件设置了一个 Ring Buffer，在 Ring Buffer 中的每一个 Entry 对应一个 DMA 描述符。使用该技术相当于在网卡中设置了多个虚拟 DMA 通路，并使用流水机制掩盖数据传送中的延时，从而提高数据传送的有效带宽。

Ring Buffer 机制还可以进一步升级为 List/Ring Buffer 机制。所谓 List/Ring Buffer 机制是为发送部件和接收部件设置多个 Ring Buffer，从而进一步提高物理链路的利用率。读者可参阅 e1000e 系列的网卡，或者 PowerPC 处理器的 TSEC 控制器，以了解 Ring Buffer 和 List/Ring Buffer 的实现机制。本节对此不做进一步描述。但是无论使用 Ring Buffer 还是 List/Ring Buffer 机制都很难进一步提高基于 PCIe 总线的网卡的数据传送率。

处理器可以根据用途分为 Control-Plane 处理器和 Data-Plane 处理器，PCIe 总线的主要功能是作为 Control-Plane 处理器的局部总线，并不是 Data-Plane 处理器的局部总线。从体系

结构的角度来看，基于 PCIe 总线的网络设备远不能与 Data-Plane 处理器的网络设备在传送效率，尤其是小报文的传送效率上，一较高低。

12.5 小结

本章通过 Capric 卡的设计实例，讲述了在 PCIe 体系结构中，事务层的硬件实现和 Linux 设备驱动程序的编写方法。其中与 Cache 一致性相关的话题最为重要，虽然这部分内容对于 PCIe 总线并不重要，但是系统程序员需要深入理解相关的概念。

本章的最后，简单分析了影响 PCIe 总线传送效率的因素，理解这些内容可以使读者在系统设计中，选择最合适的局部总线，以构建一个合理的处理器系统。值得注意的是，不同的局部总线所适用的应用领域并不相同。